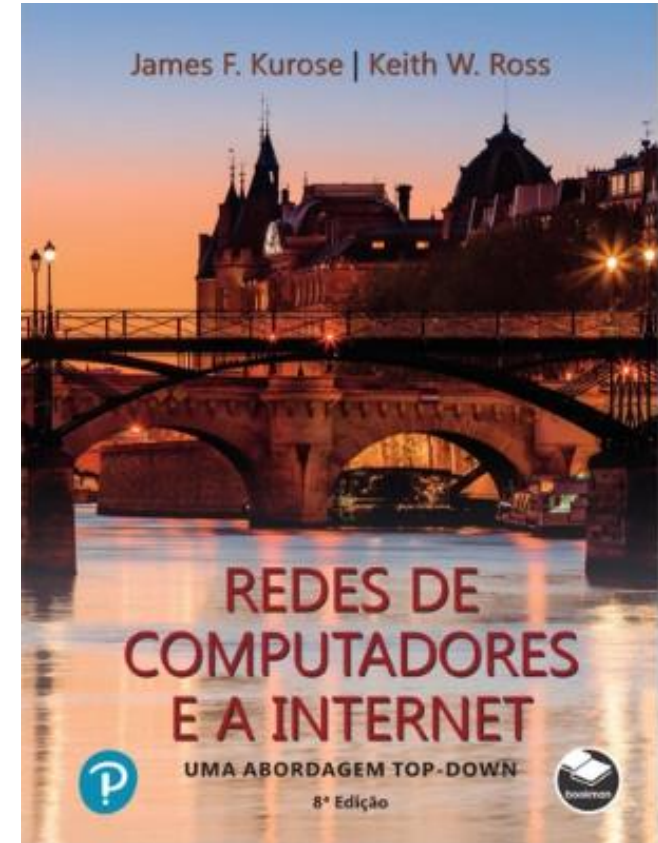


Capítulo 6

A Camada de Enlace e as LANs

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved



*Redes de Computadores e
a Internet: Uma
abordagem Top-Down*

8ª edição

Jim Kurose, Keith Ross

Pearson & Bookman, 2021

A Camada de Enlace e as LANs: nossos objetivos

- Entender os princípios por trás dos serviços da camada de enlace de dados:
 - detecção e correção de erros
 - compartilhamento de canal de difusão: acesso múltiplo
 - endereçamento da camada de enlace
 - redes locais (LANs): Ethernet, VLANs
- redes de datacenters
- instanciação e implementação de diversas tecnologias de camada de enlace



Camada de Enlace, LANs: roteiro

- **introdução**
- detecção, correção de erros
- protocolos de acesso múltiplo
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter



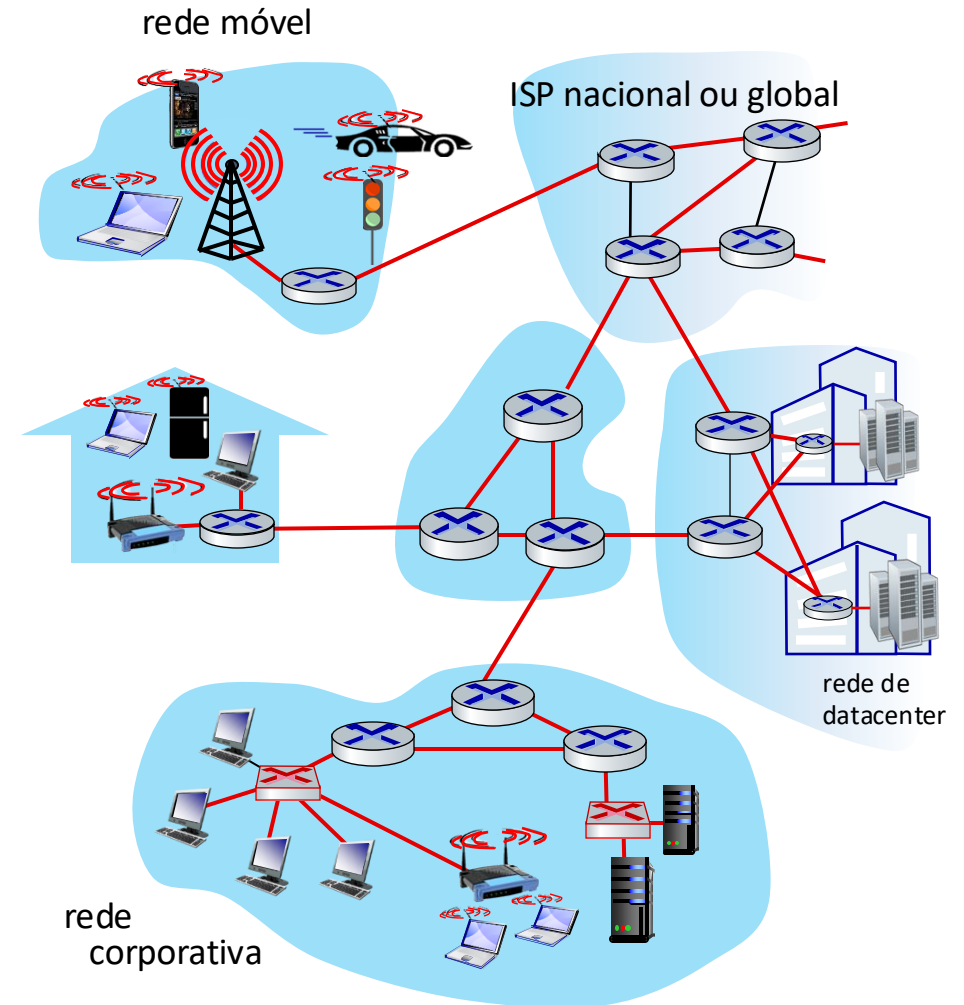
- um dia na vida de uma solicitação web

Camada de enlace: introdução

terminologia:

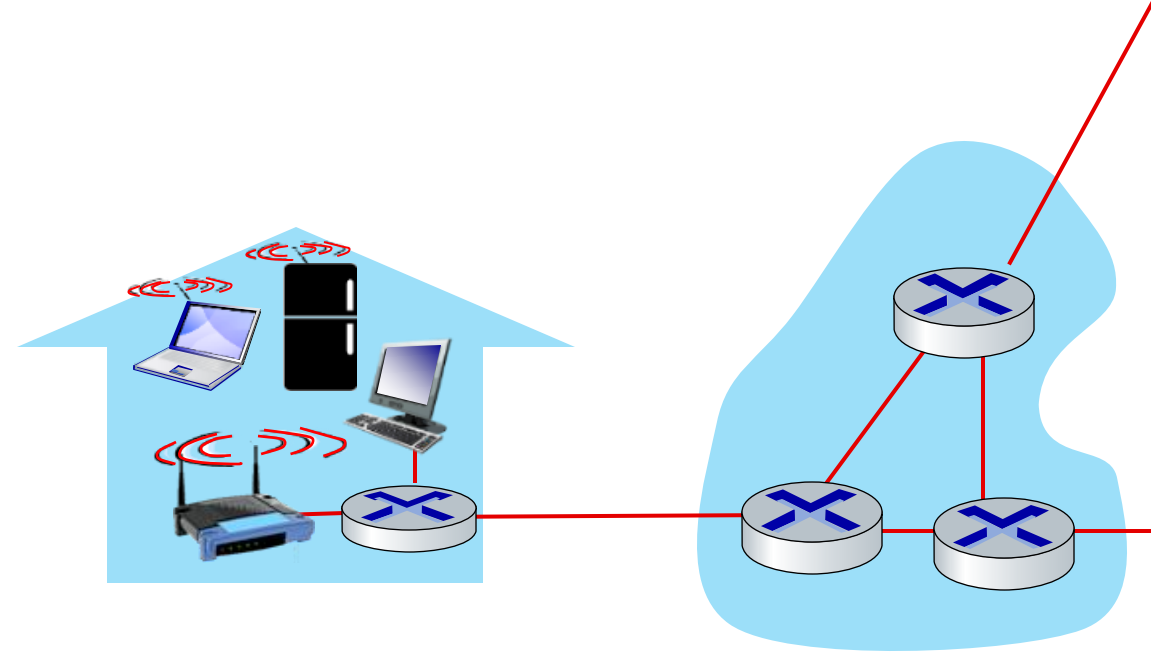
- hosts e roteadores: **nós**
- canais de comunicação que conectam nós **adjacentes** ao longo de um caminho de comunicação: **enlaces**
 - cabeados (*wired*), sem fio (*wireless*)
 - LANs
- pacote da camada-2: **quadro**, encapsula o datagrama

camada de enlace é responsável por transferir o datagrama de um nó para outro nó fisicamente adjacente através de um enlace

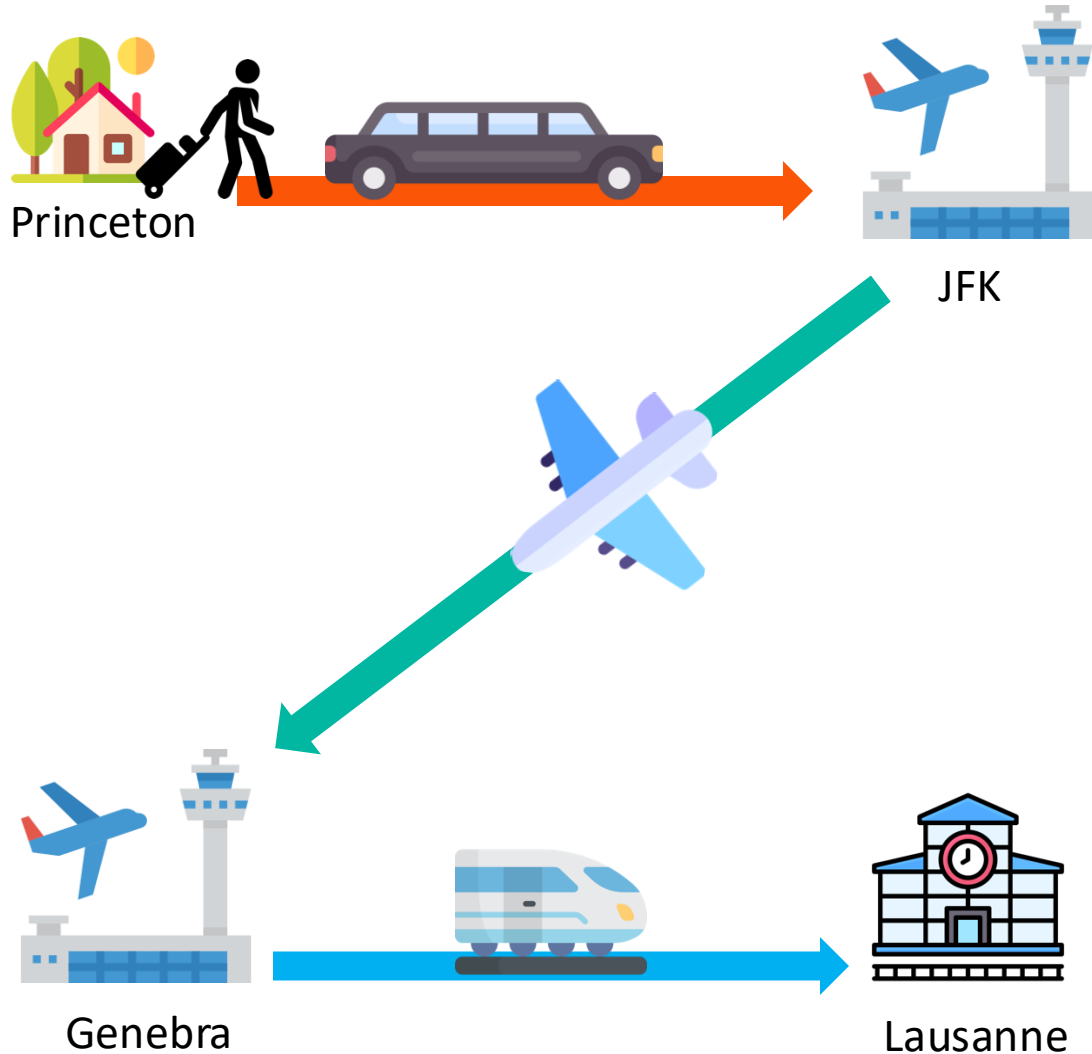


Camada de enlace: contexto

- datagrama é transferido por **diferentes protocolos de enlace** em diferentes enlaces:
 - ex., WiFi no primeiro enlace, Ethernet no próximo enlace
- cada protocolo de enlace provê diferentes serviços
 - ex.: **pode prover ou não** transporte confiável de dados através do enlace



Analogia de Transporte

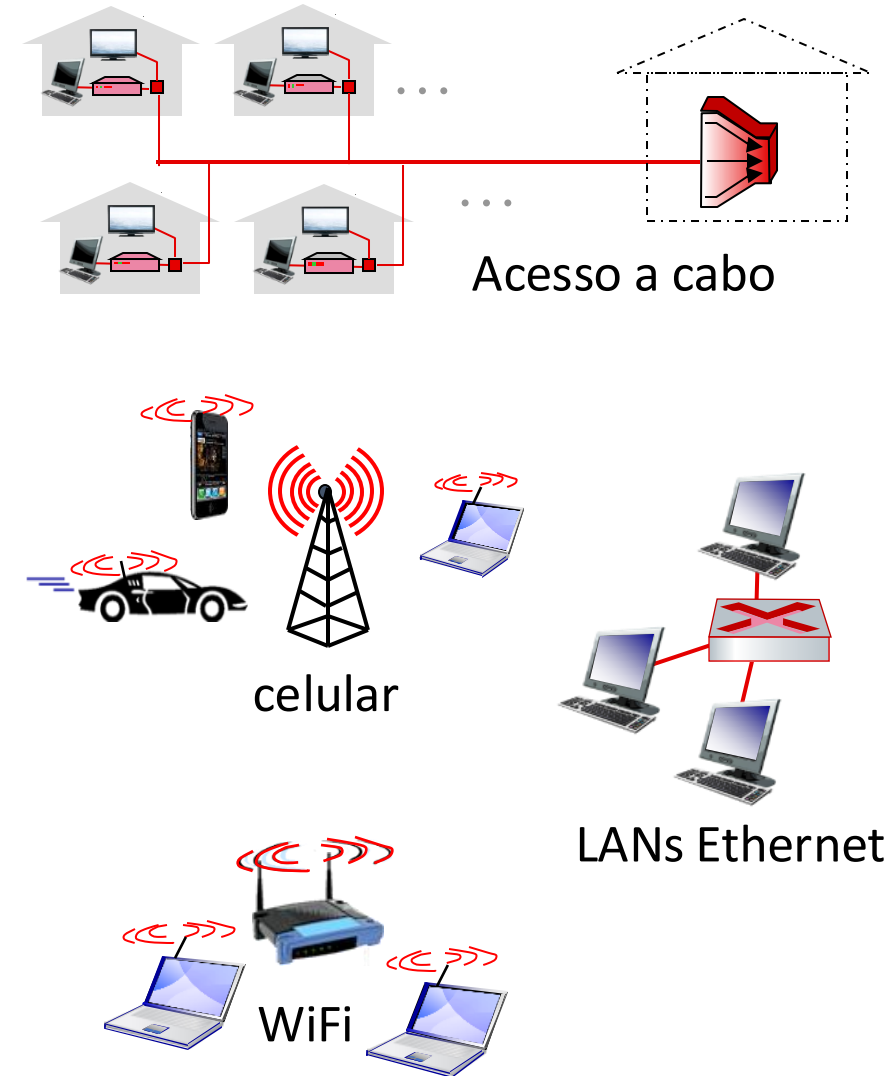


analogia de transporte:

- viagem de Princeton a Lausanne
 - taxi: Princeton até JFK
 - avião: JFK até Genebra
 - trem: Genebra até Lausanne
- turista = datagrama
- segmento de transporte = enlace de comunicação
- modo de transporte = protocolo da camada de enlace
- agente de viagem = algoritmo de roteamento

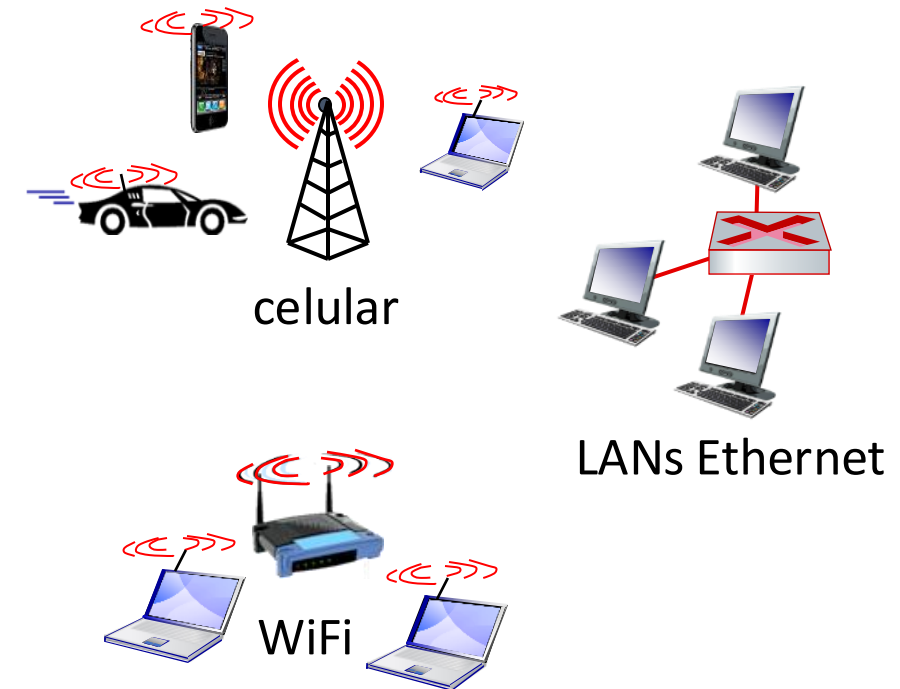
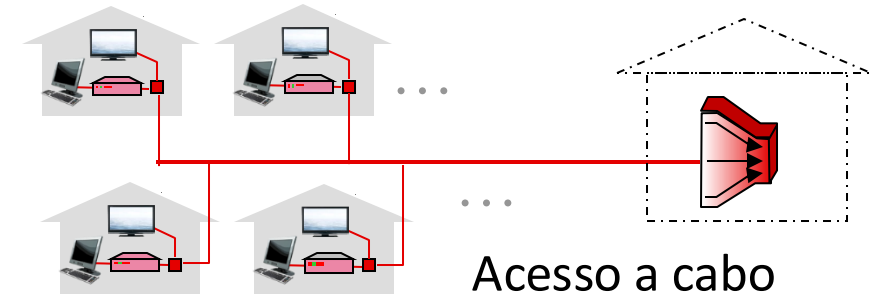
Camada de enlace: serviços

- **enquadramento, acesso ao enlace:**
 - encapsula o datagrama num quadro, adicionando cabeçalho, cauda
 - acesso ao canal se o meio for compartilhado
 - endereços “MAC” nos cabeçalhos dos quadros identificam origem, destino (diferente dos endereços IP!)
- **entrega confiável entre nós vizinhos**
 - já sabemos como fazer isso!
 - raramente usada em enlaces com baixa taxa de erros
 - enlaces sem fio: altas taxas de erros
 - Q: para que confiabilidade tanto na camada de enlace como fim-a-fim?



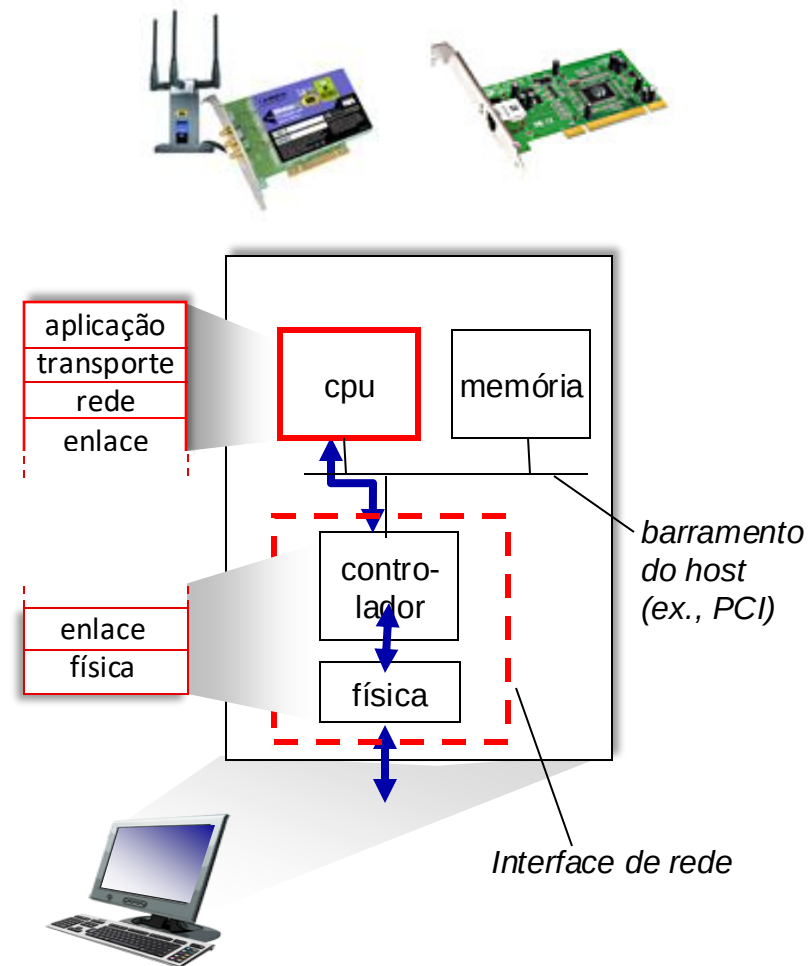
Camada de enlace: serviços (mais)

- **controle de fluxo:**
 - compatibilizar taxas entre nós transmissores e receptores adjacentes
- **detecção de erro:**
 - erros causados por atenuação do sinal, ruído.
 - receptor detecta erros, sinaliza retransmissão, ou descarta quadro
- **correção de erro:**
 - receptor identifica *e corrige* bits com erros sem retransmissão
- **half-duplex e full-duplex:**
 - no *half duplex*, os nós das extremidades do enlace podem transmitir, mas não ao mesmo tempo

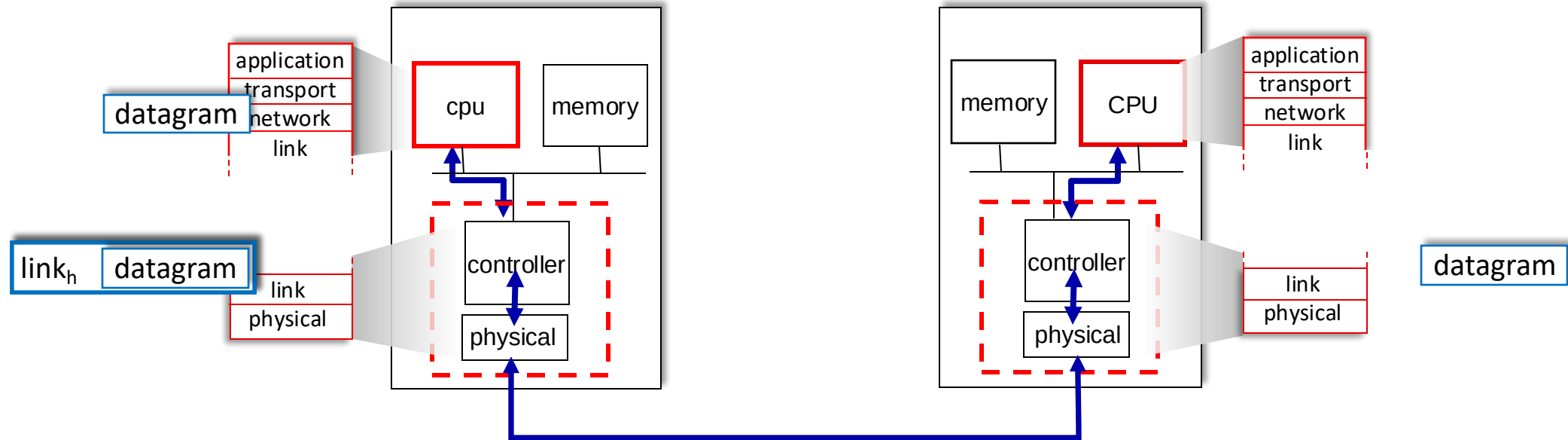


Onde é implementada a camada de enlace?

- em cada um dos nós
- camada de enlace implementada no adaptador (*network interface card* - NIC) ou em um chip
 - cartão ou chip Ethernet, WiFi
 - implementa camadas de enlace, física
- conecta ao barramento do sistema hospedeiro
- combinação de hardware, software e firmware



Comunicação entre interfaces



lado transmissor:

- encapsula datagrama no quadro
- adiciona bits de verificação de erros, transferência confiável de dados, controle de fluxo, etc.

lado receptor:

- verifica erros, transferência confiável de dados, controle de fluxo, etc.
- extrai datagrama, passa para a camada superior

Camada de Enlace, LANs: roteiro

- introdução
- **detecção, correção de erros**
- protocolos de acesso múltiplo
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter

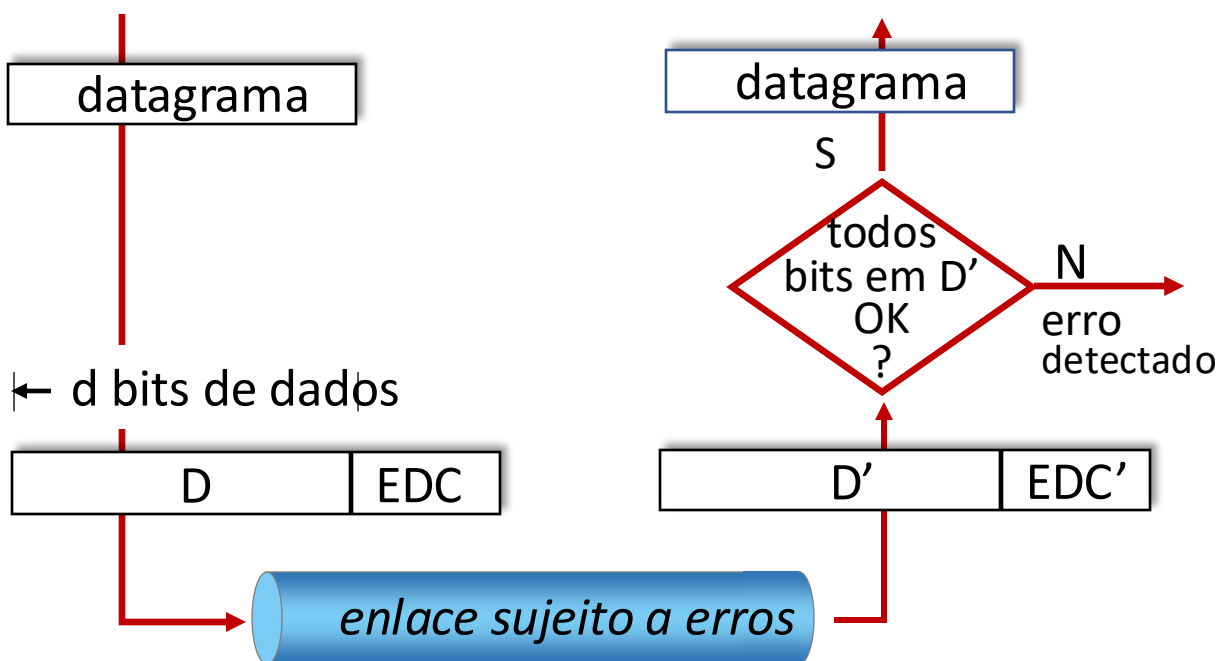


- um dia na vida de uma solicitação web

Detecção de erros

EDC: bits de detecção e correção de erros (ex., redundância)

D: dados protegidos pela verificação de erro, pode incluir campos do cabeçalho



Detecção de erro não é 100% confiável!

- protocolo pode deixar passar algum erro, mas é raro
- campos EDC maiores resultam em melhor detecção e correção

Verificação de paridade

bit único de paridade:

- detecta erros individuais

0111000110101011	1
------------------	---

← d bits de dados → | bit de paridade

Paridade par/ímpar: seta bit de paridade de modo que haja um número par/ímpar de 1's

No receptor:

- calcula a paridade dos d bits recebidos
 - Compara com o bit de paridade recebido
- erro detectado se forem diferentes



Pode detectar **e** corrigir erros (sem retransmissão!)

- bit de paridade bidimensional: detecta **e corrige** erros individuais de bits

			paridade de linha →
	$d_{1,1}$	...	$d_{1,j}$ $d_{1,j+1}$
	$d_{2,1}$	...	$d_{2,j}$ $d_{2,j+1}$
	...	...	...
	$d_{i,1}$	...	$d_{i,j}$ $d_{i,j+1}$
paridade de coluna ↓	$d_{i+1,1}$	...	$d_{i+1,j}$ $d_{i+1,j+1}$

sem erro:

1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
1	0	1	0	1	0

erro individual de bit detectado e corrigido:

1	0	1	0	1	1
1	0	1	1	0	0
0	1	1	1	0	1
1	0	1	0	1	0

erro de paridade

erro de paridade

Checksum Internet (revisão)

Objetivo: detectar erros (*i.e.*, bits trocados) no segmento transmitido

transmissor:

- trata conteúdo do segmento UDP (incluindo campos do cabeçalho UDP e endereços IP) como uma sequência de inteiros de 16-bits
- **checksum:** adição (complemento de um) do conteúdo do segmento
- valor do *checksum* colocado no campo de *checksum* do UDP

receptor:

- calcula o *checksum* do segmento recebido
- verifica se o *checksum* calculado é igual ao valor do campo de *checksum*:
 - diferente - erro detectado
 - igual - nenhum erro detectado. *Mas, ainda pode conter erros? Mais, posteriormente*

Verificação de Redundância Cíclica (CRC)

- codificação mais poderosa para a detecção de erros
- **D**: bits de dados (dado, pense nele como um número binário)
- **G**: padrão de bits (gerador), de $r+1$ bits (dado)



objetivo: escolher r bits CRC, **R**, de modo que $\langle D, R \rangle$ seja exatamente divisível por $G \pmod{2}$

- receptor conhece G , divide $\langle D, R \rangle$ por G . Se o resto não for zero: erro detectado!
- pode detectar todos os erros em rajadas menores do que $r+1$ bits
- largamente usado na prática (Ethernet, 802.11 WiFi)

Verificação de Redundância Cíclica (CRC): exemplo

Queremos:

$$D \cdot 2^r \text{ XOR } R = nG$$

ou de modo equivalente:

$$D \cdot 2^r = nG \text{ XOR } R$$

de forma equivalente:

se dividirmos $D \cdot 2^r$ por G ,
queremos o resto R que
satisfaça:

$$R = \text{resto} \left(\frac{D \cdot 2^r}{G} \right)$$

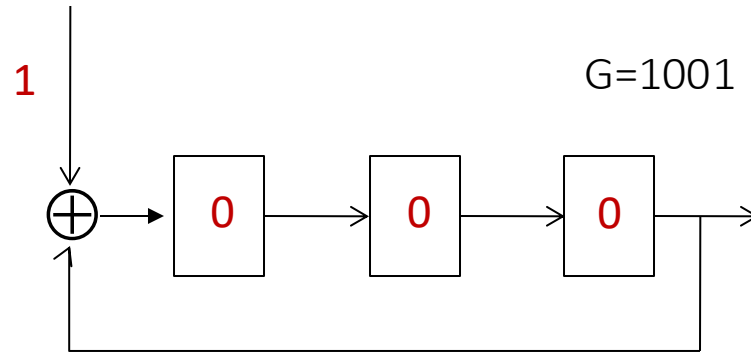
*algoritmo para
calcular R*

$$G(x) = 1 \cdot x^3 + 0 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0 = x^3 + 1 \quad r = 3$$

D		G
101110	000	1001
1001		101011
1010		
1001		
1100		
1001		
1010		
1001		
011		
		R

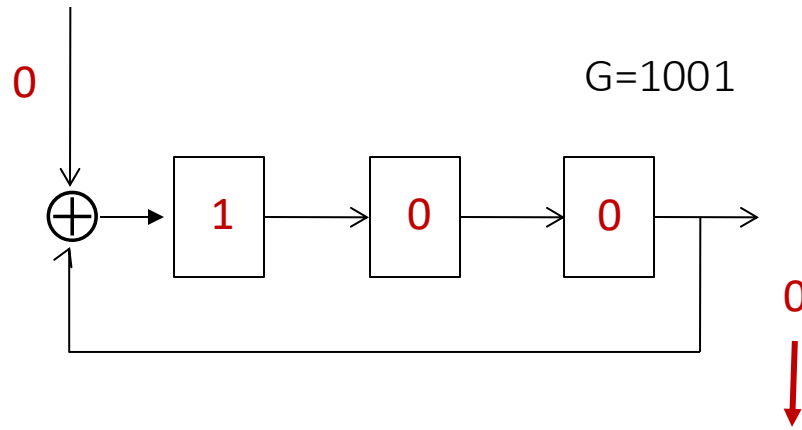
Implementação em Hardware

$D \cdot 2^r = 101110000$



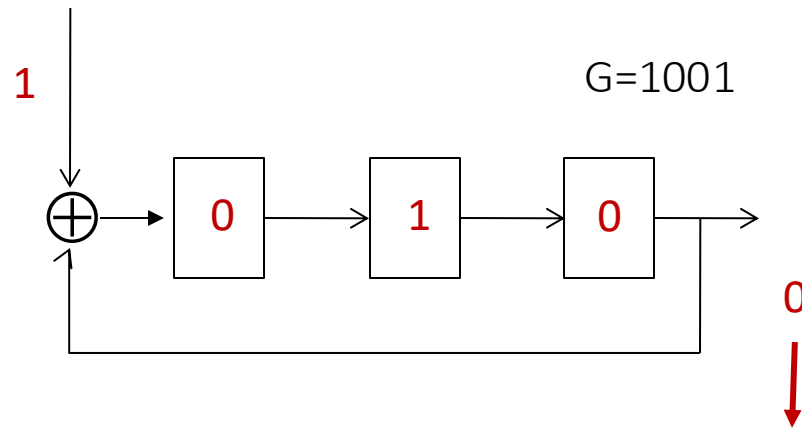
Implementação em Hardware

$D \cdot 2^r = 101110000$



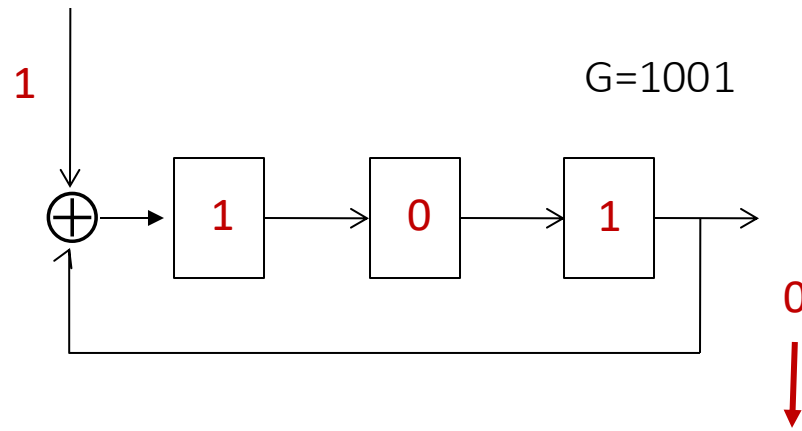
Implementação em Hardware

$D \cdot 2^r = 101110000$



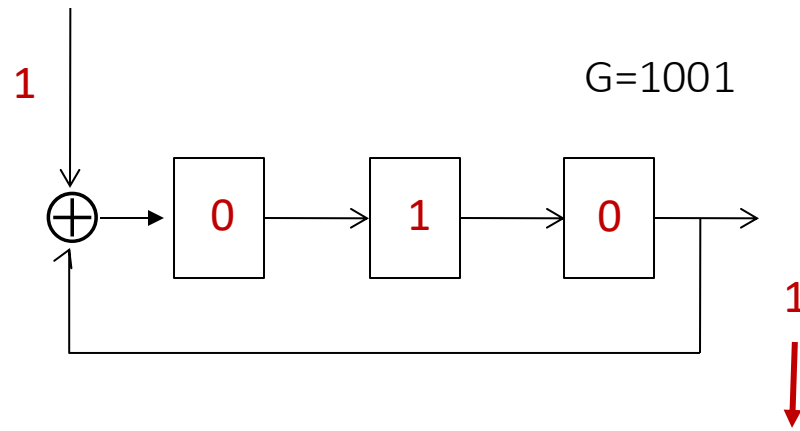
Implementação em Hardware

$D \cdot 2^r = 101110000$



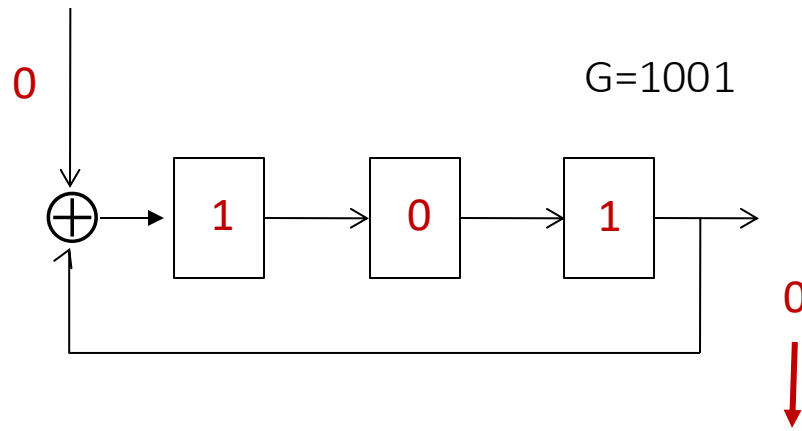
Implementação em Hardware

$D \cdot 2^r = 101110000$



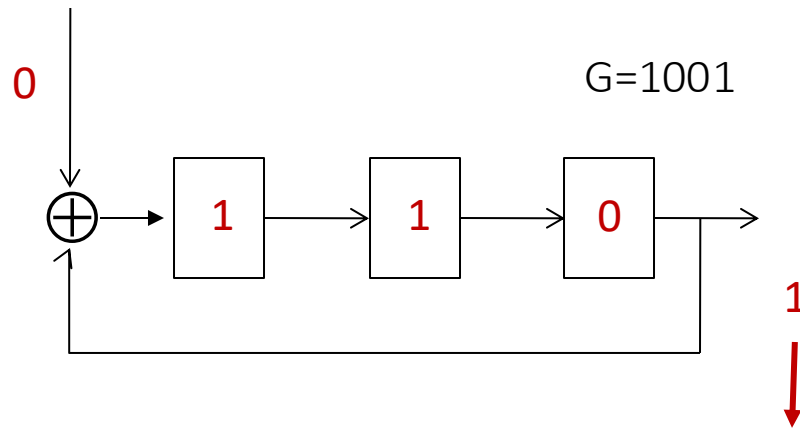
Implementação em Hardware

$D \cdot 2^r = 101110000$



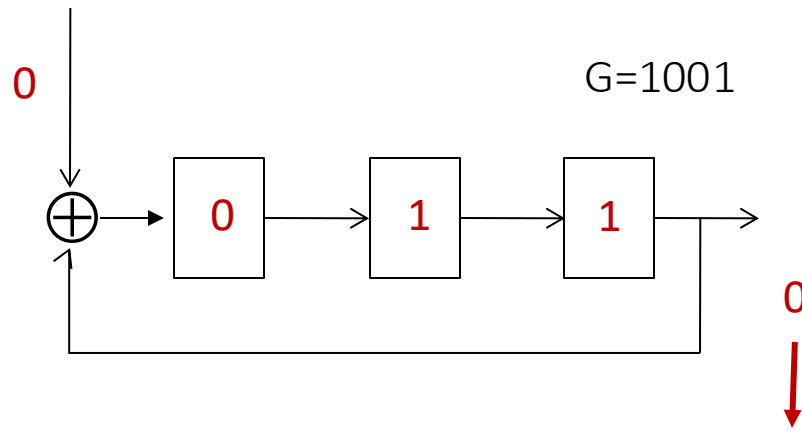
Implementação em Hardware

$D \cdot 2^r = 101110000$



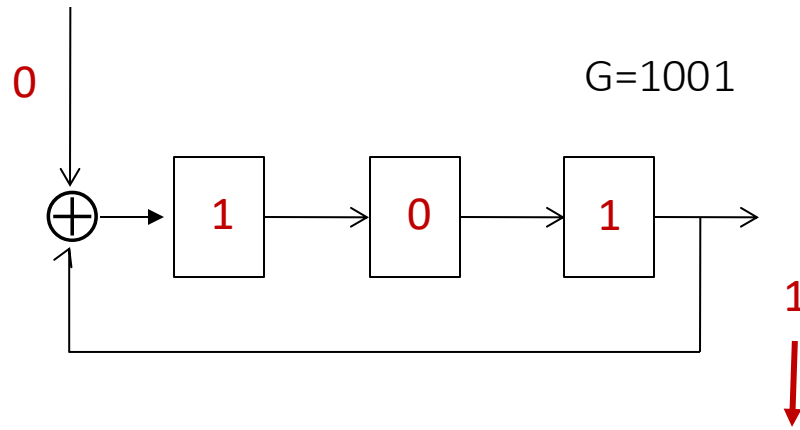
Implementação em Hardware

$D \cdot 2^r = 101110000$



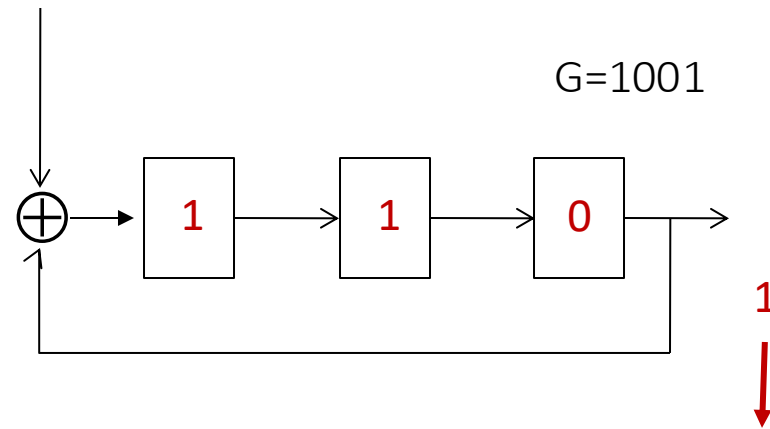
Implementação em Hardware

$D \cdot 2^r = 101110000$



Implementação em Hardware

$D \cdot 2^r = 101110000$



Resto da divisão, da direita
para a esquerda!

Camada de Enlace, LANs: roteiro

- introdução
- detecção, correção de erros
- **protocolos de acesso múltiplo**
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter



- um dia na vida de uma solicitação web

Enlaces, protocolos de acceso múltiplo

dois tipos de “enlaces”:

- **ponto-a-ponto**
 - enlace ponto-a-ponto entre switch Ethernet, host
 - PPP para acesso discado
- **difusão (cabo ou meio compartilhado)**
 - Ethernet clássico
 - *upstream* HFC em rede de acesso baseada em cabo
 - LAN sem fio 802.11, 4G/5G, satélite



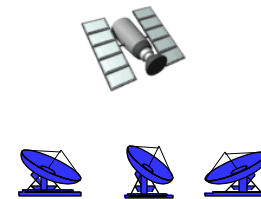
cabo compartilhado
(ex., Ethernet
cabeado)



rádio compartilhado:
4G/5G



rádio compartilhado:
WiFi



rádio compartilhado:
Satélite



peças num coquetel
(ar, acústica compartilhados)

Protocolos de acesso múltiplo

- um único canal de difusão compartilhado
- duas ou mais transmissões simultâneas pelos nós: interferência
 - *colisão* se um nó receber dois ou mais sinais ao mesmo tempo

Protocolo de acesso múltiplo

- algoritmo distribuído que determina como os nós compartilham o canal, isto é, determina quando um nó pode transmitir
- comunicação sobre o compartilhamento do canal deve usar o próprio canal!
 - não há canal fora da faixa para coordenar a transmissão

Um protocolo ideal de acesso múltiplo

dado: canal de múltiplo acesso (MAC) com taxa de R bps

desejamos:

1. quando apenas um nó quiser transmitir, pode transmitir com taxa R .
2. quando M nós quiserem transmitir, cada um poderá enviar a uma taxa de R/M
3. completamente descentralizado:
 - nenhum nó especial (mestre) para coordenar as transmissões
 - nenhuma sincronização de relógios ou slots
4. simples

Protocolos MAC: taxonomia

três classes gerais:

- **divisão de canal**

- divide canal em pequenos “pedaços” (compartimentos de tempo, frequência, código)
- aloca pedaço para uso exclusivo de um nó

- ***acesso aleatório***

- canal não é dividido, podem ocorrer colisões
- “recuperação” após colisões

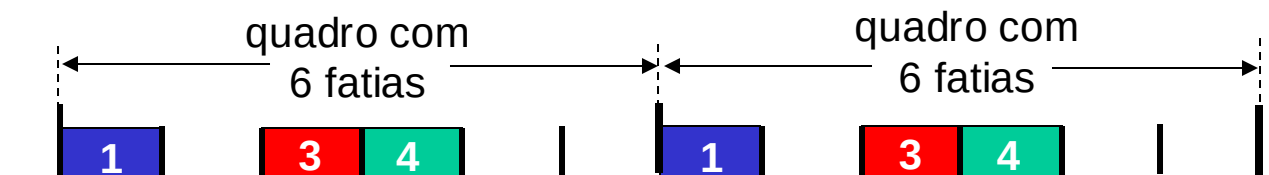
- **revezamento**

- nós se alternam, mas um nó que tiver mais para transmitir pode demorar mais tempo na sua vez

Protocolos MAC de divisão de canal: TDMA

TDMA: acesso múltiplo por divisão de tempo

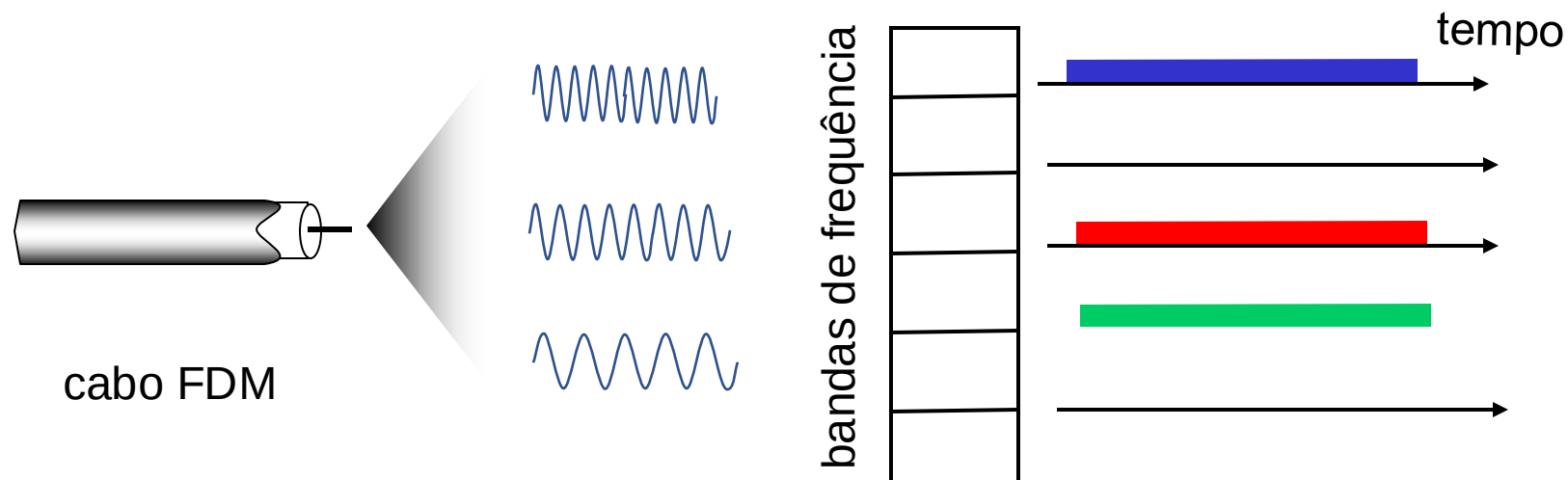
- acesso em “turnos” ao canal
- cada estação recebe uma fatia de comprimento fixo (comp. = tempo de transmissão de um pacote) a cada turno
- fatias não utilizadas ficam ociosas
- Exemplo: LAN com 6 estações, 1,3,4 têm pacote a enviar, fatias 2,5,6 ociosas



Protocolos MAC de divisão de canal: FDMA

FDMA: acesso múltiplo por divisão de frequência

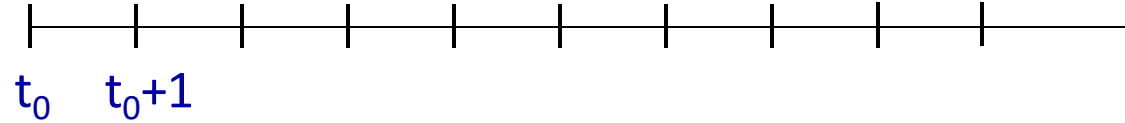
- espectro do canal dividido em bandas de frequência
- a cada estação é atribuída uma banda fixa de frequência
- tempo de transmissão não usado nas bandas permanecem ociosos
- exemplo: LAN com 6 estações, 1,3,4 com pacotes, bandas 2,5,6 ociosas



Protocolos de acesso aleatório

- quando nó tem um pacote para transmitir
 - transmite na taxa máxima R .
 - nenhuma coordenação a priori entre os nós
- dois ou mais nós transmitindo: “colisão”
- **protocolo MAC de acesso aleatório** especifica:
 - como detectar colisões
 - como se recuperar delas (ex.: através de retransmissões retardadas)
- exemplos de protocolos MAC de acesso aleatório:
 - ALOHA, *slotted* ALOHA
 - CSMA, CSMA/CD, CSMA/CA

Slotted ALOHA



hipóteses:

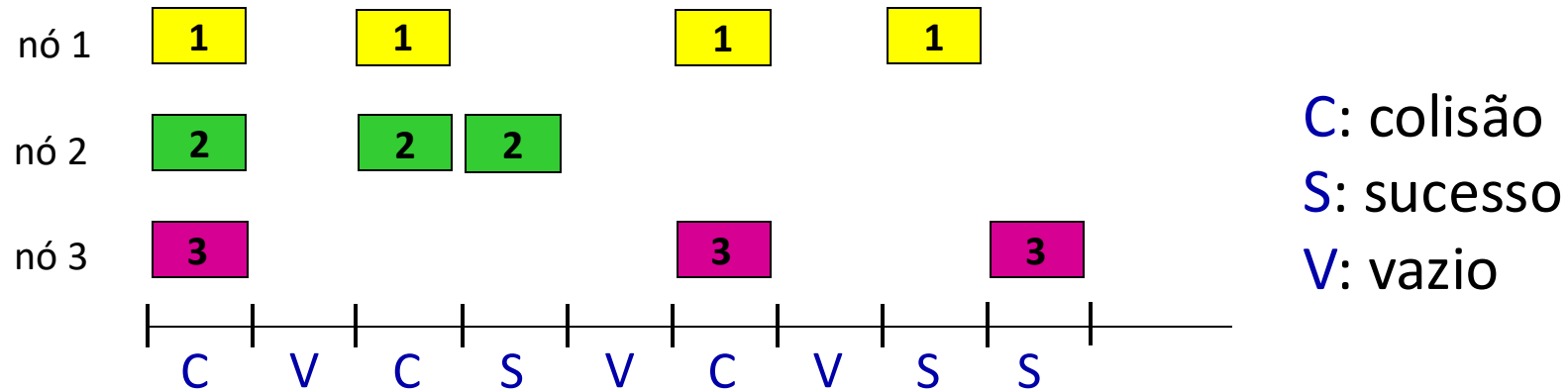
- todos os quadros do mesmo tamanho
- Tempo dividido em *slots* de comprimento igual (tempo para transmitir 1 quadro)
- nós iniciam transmissão apenas no início de um *slot*
- nós são sincronizados
- se 2 ou mais nós transmitirem num mesmo *slot*, todos os nós detectam a colisão

operação:

- quando nó obtém novo quadro, transmite no próximo *slot*
 - *se não houver colisão*: nó poderá enviar novo quadro no próximo slot
 - *se houver colisão*: nó retransmite o quadro em cada *slot* subsequente, com probabilidade p até obter sucesso

randomização – por que?

Slotted ALOHA



Vantagens:

- único nó ativo pode transmitir continuamente na taxa máxima do canal
- Altamente descentralizado: apenas slots nos nós precisam estar sincronizados
- simples

Desvantagens:

- colisões: *slots* desperdiçados
- slots ociosos (desperdício)
- nós podem ser capazes de detectar colisões num tempo inferior ao da transmissão do pacote
- sincronização dos relógios

Slotted ALOHA: eficiência

eficiência: fração de longo prazo de slots bem sucedidos (muitos nós, todos com muitos quadros para transmitir)

- *assuma:* N nós com muitos quadros para transmitir, cada um transmite num *slot* com probabilidade p .
 - prob que um dado nó tenha sucesso em um *slot* = $p(1-p)^{N-1}$
 - prob que *algum* nó tenha sucesso = $Np(1-p)^{N-1}$
 - eficiência máx: encontre p^* que maximize $Np(1-p)^{N-1}$
 - Para muitos nós, passar o limite de $Np^*(1-p^*)^{N-1}$ quando N vai para infinito, dá:

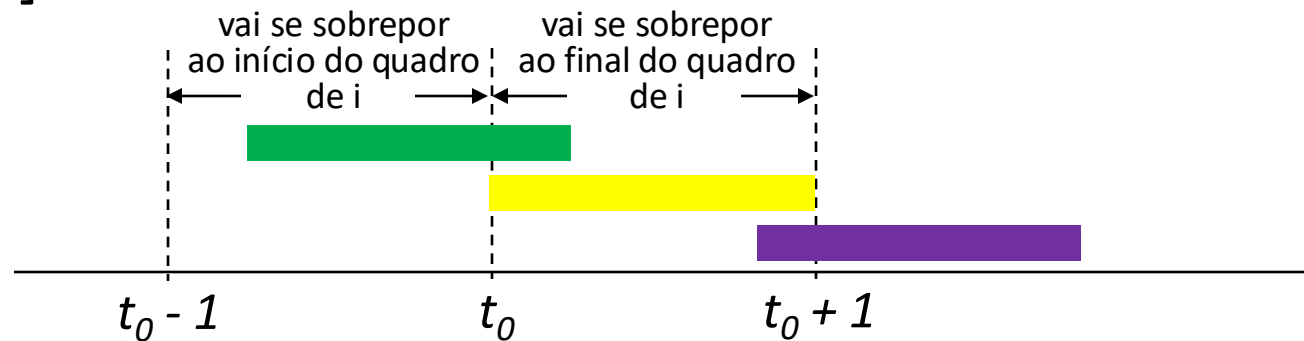
eficiência máx = $1/e = 0,37$

- *melhor caso:* canal usado para transmissões úteis apenas 37% do tempo!



ALOHA puro

- Aloha sem *slots*: mais simples, sem sincronização
 - quando quadro chegar: transmite imediatamente
- probabilidade de colisão cresce com a não sincronização:
 - quadro enviado em t_0 colide com outros quadros enviados em $[t_0-1, t_0+1]$



- eficiência do Aloha puro: 18% !

CSMA (*carrier sense multiple access*)

CSMA simples: escuta antes de transmitir:

- se não for detectada transmissão: transmite todo o quadro
 - se o canal estiver ocupado: adia a transmissão
- analogia humana: não interrompa os outros!

CSMA/CD: CSMA com *detecção de colisões*

- colisões *detectadas* em pouco tempo
 - transmissões que colidiram são abortadas, reduzindo o desperdício do canal
 - detecção de colisões fácil em redes cabeadas, difícil em redes sem-fio
- analogia humana: bate papo educado

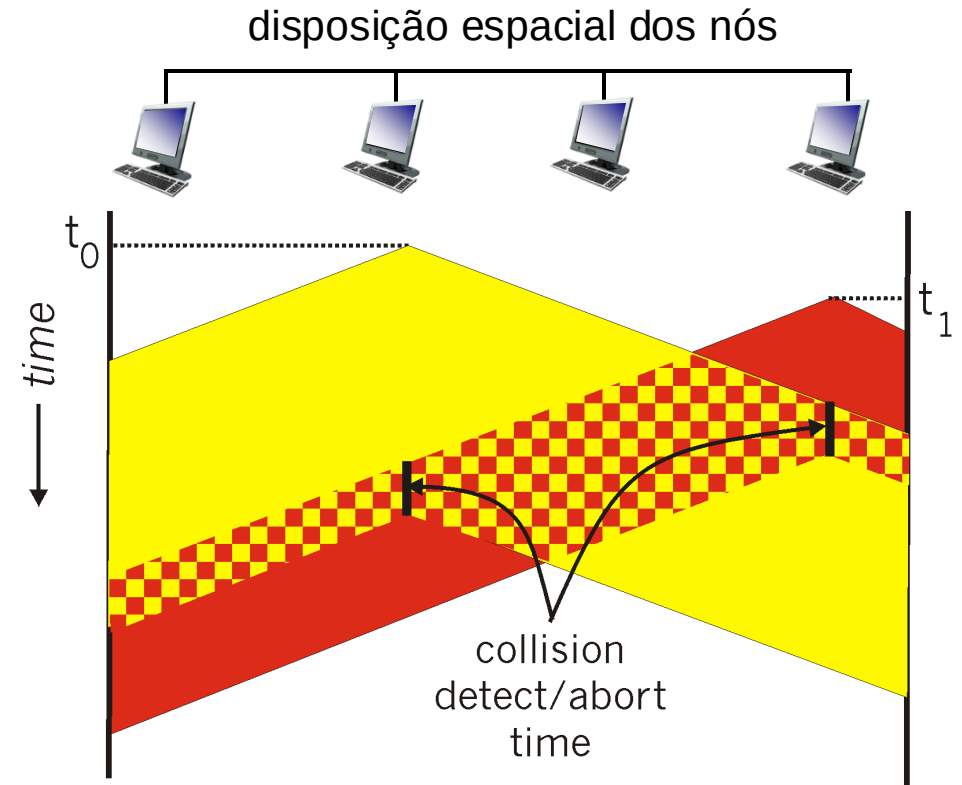
CSMA: colisões

- colisões ainda podem ocorrer mesmo com detecção de portadora:
 - atraso de propagação implica em dois nós não conseguirem ouvir imediatamente a transmissão um do outro
- **colisão**: todo o tempo de transmissão dos pacotes foi desperdiçado
 - distância & atraso de propagação têm um papel na determinação da probabilidade de colisão



CSMA/CD:

- CSMA/CD reduz a quantidade de tempo desperdiçado com as colisões
 - transmissão abortada ao detectar a colisão



Algoritmo Ethernet CSMA/CD

1. Cartão (NIC) recebe **datagrama** da camada de rede, cria o quadro
2. Se o cartão percebe que:
 - canal **livre**: inicia transmissão do quadro.
 - canal **ocupado**: espera até que o canal fique livre, então transmite
3. Se o cartão transmitir todo quadro sem colisão, cartão encerrou com o quadro!
4. Se o cartão detectar outra transmissão enquanto estiver transmitindo: aborta, envia sinal de reforço
5. Após abortar, cartão entra na *retirada (exponencial) binária*:
 - após a m -ésima colisão, cartão escolhe K aleatoriamente entre $\{0, 1, 2, \dots, 2^m - 1\}$. cartão espera $K \cdot 512$ tempos de bit, retorna ao Passo 2
 - mais colisões: aumenta os intervalos de retirada

CSMA/CD: eficiência

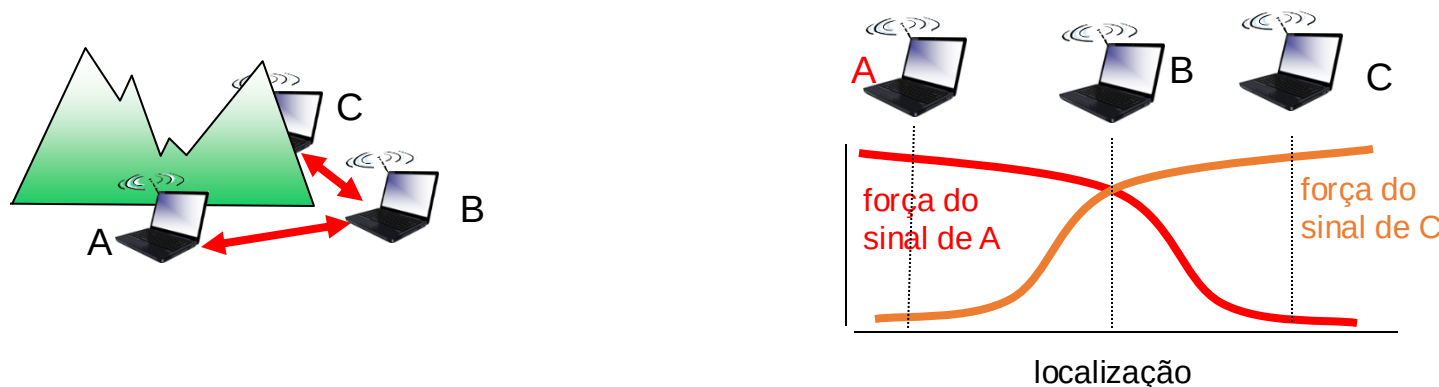
- t_{prop} = atraso máximo de propagação entre 2 nós na LAN
- t_{trans} = tempo para transmitir quadro de tamanho máximo

$$\text{eficiência} = \frac{1}{1 + 5t_{prop}/t_{trans}}$$

- eficiência vai para 1 à medida que:
 - t_{prop} vai para 0 (os nós sabem logo de alguma transmissão)
 - t_{trans} vai para infinito (reduz o número de novas tentativas!)
- melhor desempenho que o ALOHA: e ainda é simples, barato, e descentralizado!

IEEE 802.11: acesso múltiplo

- Evita colisões: 2 ou mais nós transmitindo ao mesmo tempo
- 802.11: CSMA – escuta antes de transmitir
 - Não colide com transmissões em curso de outros nós
- 802.11: não faz detecção de colisão!
 - Difícil de receber (sentir as colisões) quando transmitindo devido ao fraco sinal recebido (desvanecimento)
 - Pode não perceber as colisões: terminal oculto, fading
 - Meta: **evitar colisões**: CSMA/C(collision)A(voidance)



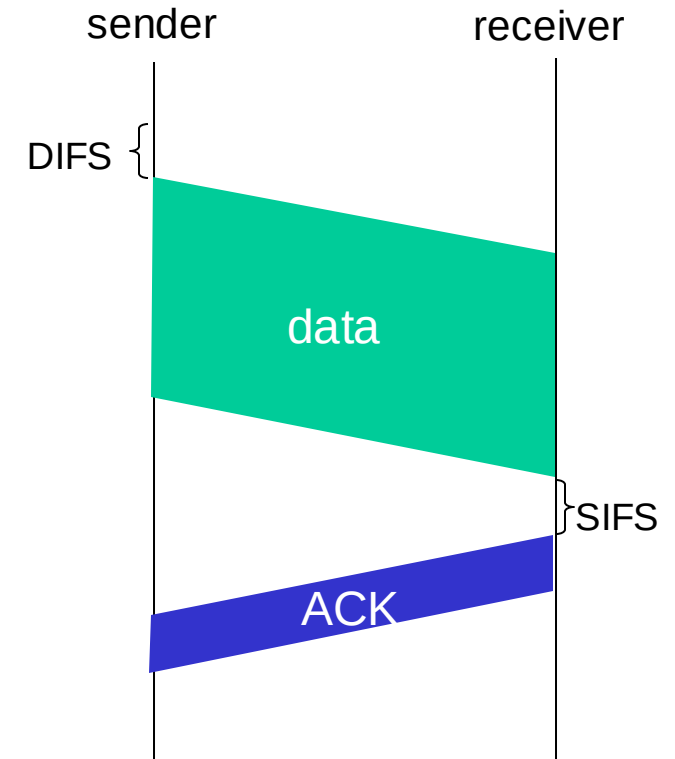
Protocolo MAC IEEE 802.11: CSMA/CA

transmissor 802.11

- 1 se o canal é percebido quieto (*idle*) por **DIFS** então transmite o quadro inteiro (sem CD)
- 2 se o canal é percebido ocupado, então inicia um tempo de *backoff* aleatório temporizador faz contagem regressiva enquanto o canal está ocioso transmite quando o temporizador expirar se não vier um ACK, aumenta o intervalo de *backoff* aleatório, repete 2

receptor 802.11

- se o quadro for recebido OK
retorna ACK após **SIFS** (ACK é necessário devido ao problema do terminal oculto)

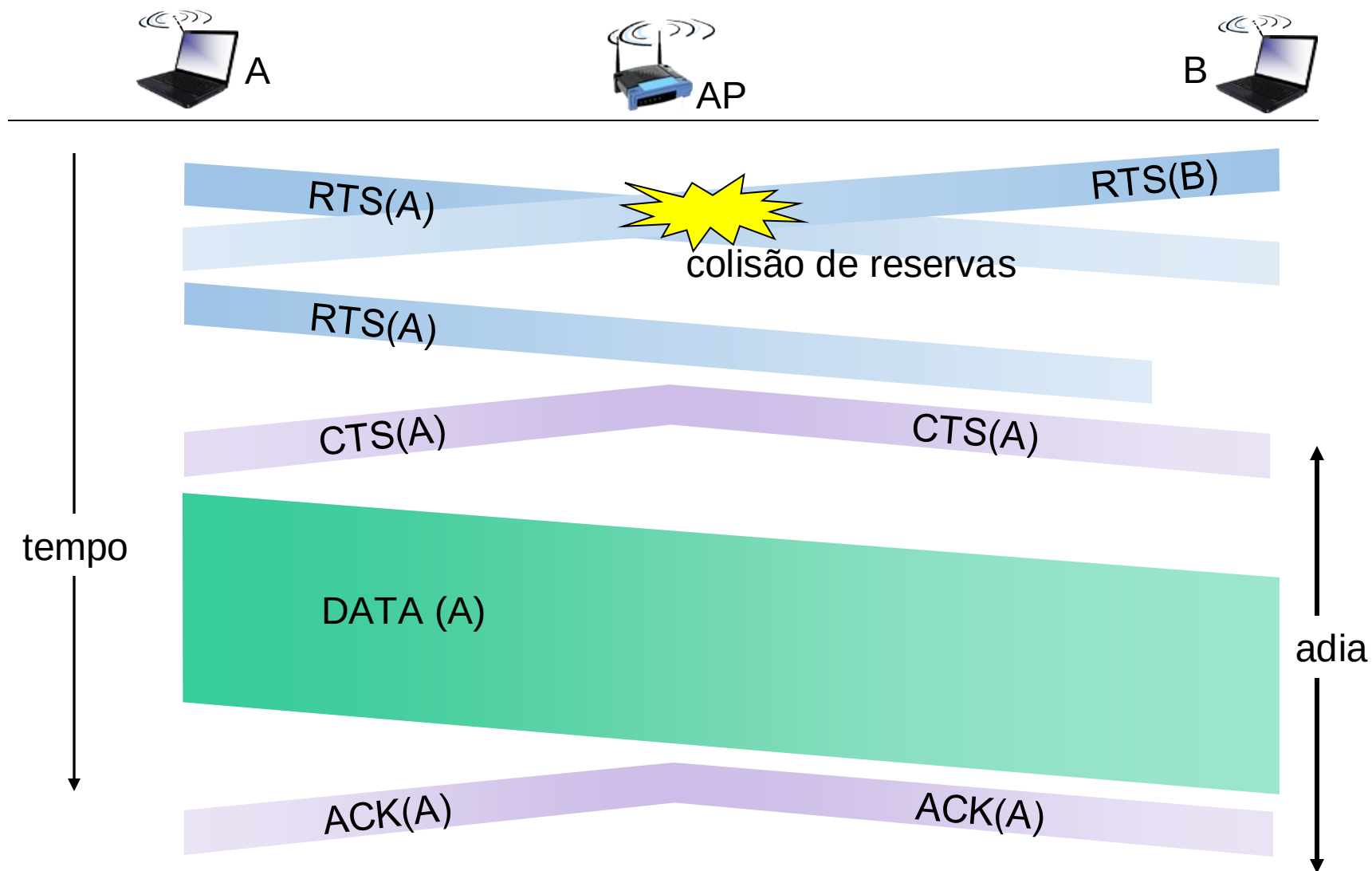


Evitando colisões (mais)

ideia: permite o transmissor “reservar” o canal em vez de acessar aleatoriamente ao enviar quadros de dados: evita colisões de quadros grandes

- transmissor envia primeiro um pequeno quadro chamado *request to send* (RTS) à estação-base usando CSMA
 - RTSs podem ainda colidir uns com os outros, mas são pequenos
- BS envia em difusão *clear to send* CTS em resposta ao RTS
- CTS é ouvido por todos os nós
 - transmissor envia o quadro de dados
 - outras estações adiam suas transmissões

Evitando colisões: reservas com RTS-CTS



Protocolos MAC

protocolos MAC de divisão de canal:

- compartilha o canal *eficientemente* e de forma *justa* em altas cargas
- ineficiente em baixas cargas: atraso no canal de acesso, alocação de $1/N$ da largura de banda mesmo com apenas 1 nó ativo!

protocolos MAC de acesso aleatório

- eficiente em baixas cargas: um único nó pode utilizar completamente o canal
- altas cargas: *overhead* com colisões

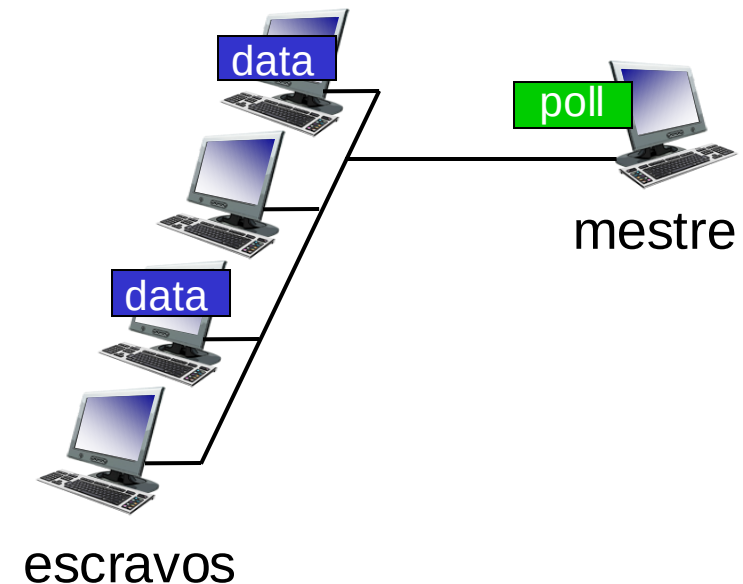
protocolos de “revezamento”

- procura oferecer o melhor dos dois mundos!

Protocolos MAC de “revezamento”

consulta (*polling*):

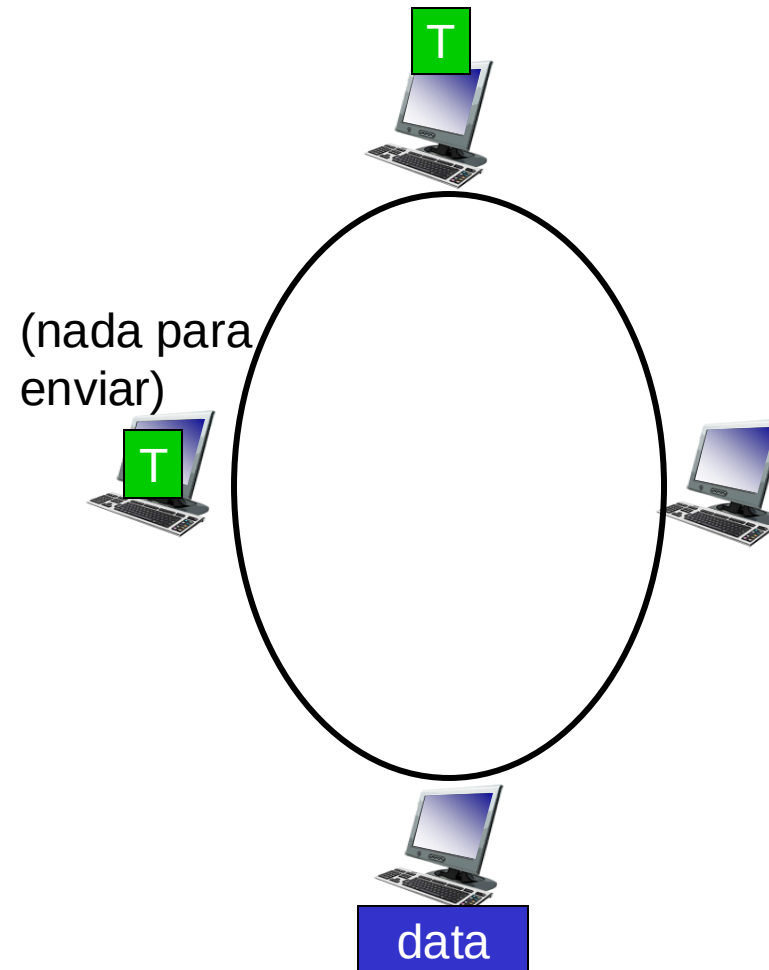
- nó mestre “convida” demais nós a transmitir em revezamento
- usado tipicamente com dispositivos “burros”
- preocupações:
 - *overhead* com as consultas
 - latência
 - ponto único de falha (mestre)



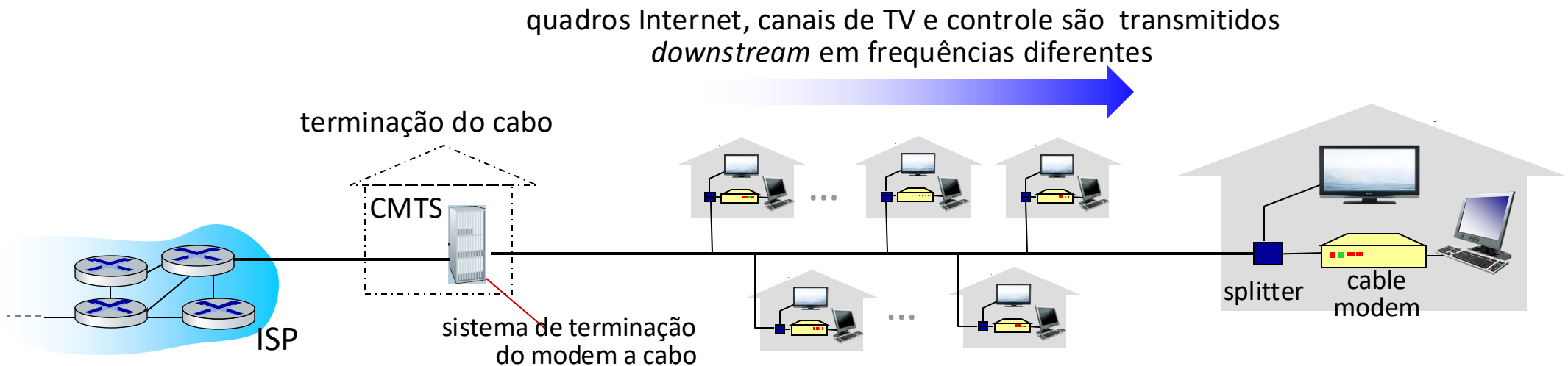
Protocolos MAC de “revezamento”

passagem de permissão (*token*):

- **permissão** de controle é passada de um nó para o próximo de forma sequencial.
- mensagem da permissão.
- preocupações:
 - *overhead* da permissão
 - latência
 - ponto único de falha (permissão)

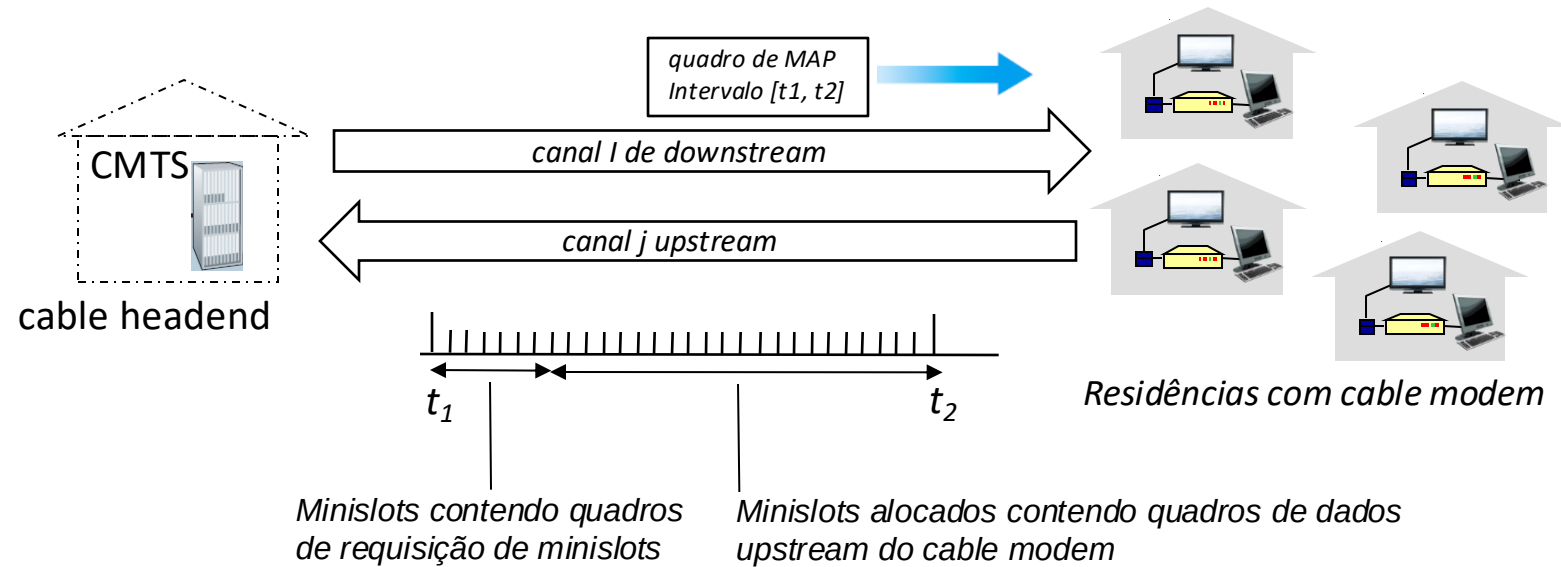


Rede de acesso a cabo: FDM, TDM e acesso aleatório!



- **múltiplos** canais FDM *downstream* (difusão): até 1,6 Gbps/canal
 - um único CMTS transmite em todos os canais
- **múltiplos** canais *upstream* (até 1 Gbps/canal)
 - **acesso múltiplo**: todos os usuários disputam (acesso aleatório) slots de tempo no canal *upstream*; outros são pré-allocados a canais TDM

Rede de acesso a cabo:



DOCSIS: *data over cable service interface specification*

- FDM nas frequências dos canais *upstream*, *downstream*
- TDM no canal *upstream*: alguns slots alocados, outros sofrem disputa
 - quadro MAP *downstream*: aloca slots *upstream*
 - requisição de slots *upstream* (e dados) são transmitidos através de acesso aleatório (retirada binária) em slots selecionados

Resumo dos protocolos MAC

- **divisão do canal**, por tempo, frequência ou código
 - Divisão de Tempo, Divisão de Frequência
- **acesso aleatório** (dinâmico),
 - ALOHA, S-ALOHA, CSMA, CSMA/CD
 - escutar a portadora: fácil em algumas tecnologias (cabeadas), difícil em outras (sem fio)
 - CSMA/CD usado na Ethernet
 - CSMA/CA usado no 802.11
- **revezamento**
 - consulta (*polling*) a partir de um ponto central, passagem de permissões
 - Bluetooth, FDDI, token ring

Camada de Enlace, LANs: roteiro

- introdução
- detecção, correção de erros
- protocolos de acesso múltiplo
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter



- um dia na vida de uma solicitação web

Endereços MAC

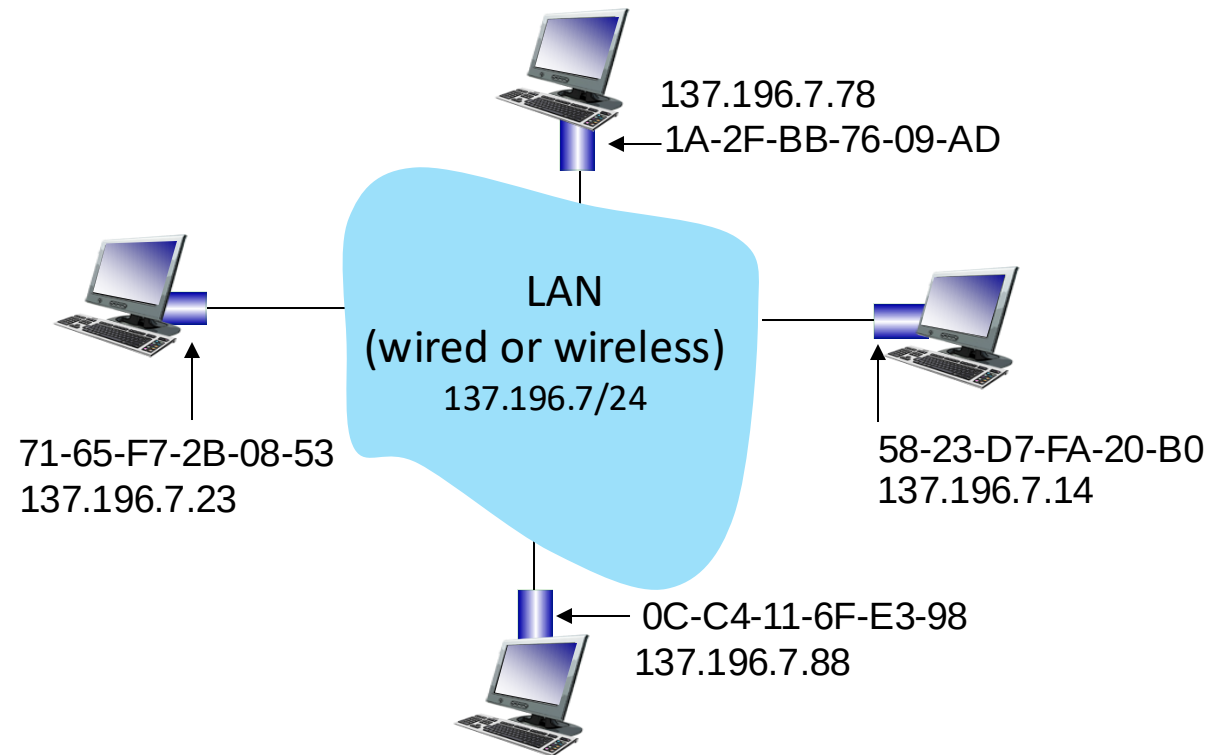
- endereços IP de 32-bits:
 - endereços da *camada de rede* para a interface
 - usado pelo repasse da camada 3 (camada de rede)
 - ex.: 128.119.40.136
- endereço MAC (ou LAN ou físico ou Ethernet):
 - função: usado “localmente” para levar o quadro de uma interface até outra interface conectada fisicamente (na mesma rede, no sentido do endereçamento IP)
 - endereço MAC de 48-bits (para muitas LANs) gravado na ROM do NIC, algumas vezes configurado por software
 - ex.: 1A-2F-BB-76-09-AD

*notação hexadecimal (base 16)
(cada “número” representa 4 bits)*

Endereços MAC

cada interface na LAN possui

- endereço **MAC** único de 48-bits
- endereço IP único localmente de 32-bits (como vimos)



Endereços MAC

- alocação de endereços MAC gerenciada pelo IEEE
- fabricantes compra uma parte do espaço de endereços MAC (para garantir unicidade)
- analogia:
 - endereço MAC: como número do CPF
 - endereço IP: como endereço postal (CEP)
- endereço MAC tem estrutura plana: portabilidade
 - pode mover interface de uma LAN para outra
 - endereço IP não é portátil: depende da subrede IP à qual o nó está conectado

ARP: *address resolution protocol*

Questão: como obter o endereço MAC da interface, a partir do seu endereço IP?

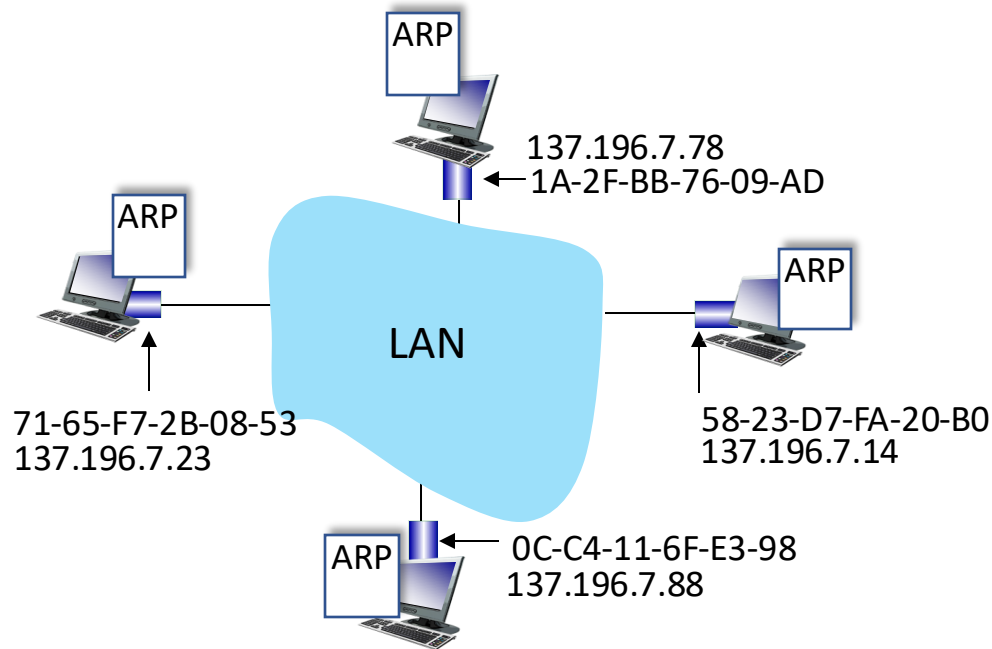


tabela ARP: cada nó IP (host, roteador) numa LAN possui a tabela

- mapeamentos de endereços IP/MAC para alguns nós da LAN:
- < endereço IP; endereço MAC; TTL >
- TTL (*Time To Live*): tempo a partir do qual o mapeamento de endereços será esquecido (valor típico de 20 min)

Protocolo ARP em ação

exemplo: A deseja enviar datagrama para B

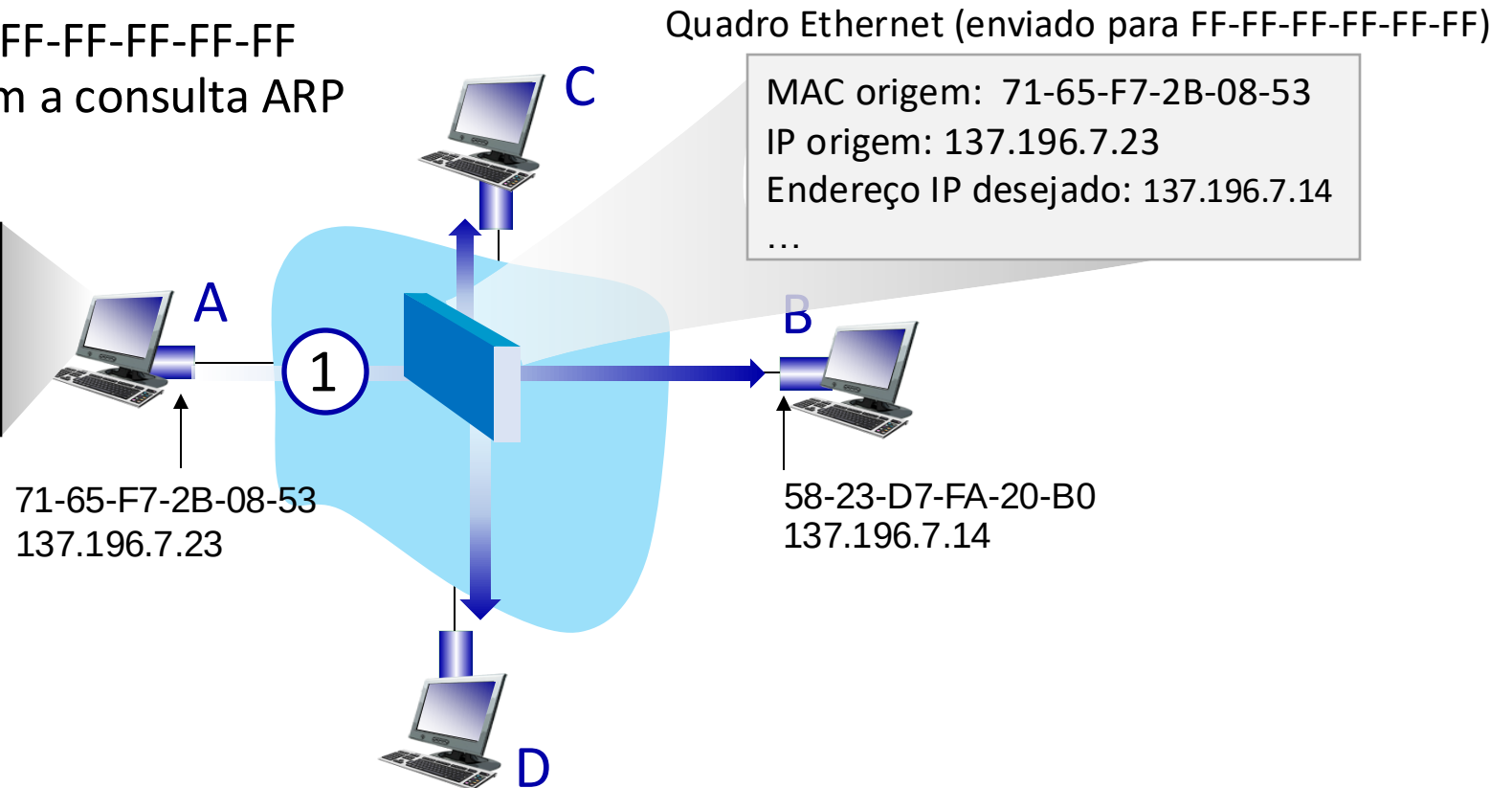
- endereço MAC de B não está na tabela ARP de A, então A usa ARP para obter o endereço MAC de B

A difunde consulta ARP, contendo end. IP de B

- ①
- endereço MAC destino = FF-FF-FF-FF-FF-FF
 - todos os nós na LAN recebem a consulta ARP

Tabela ARP em A

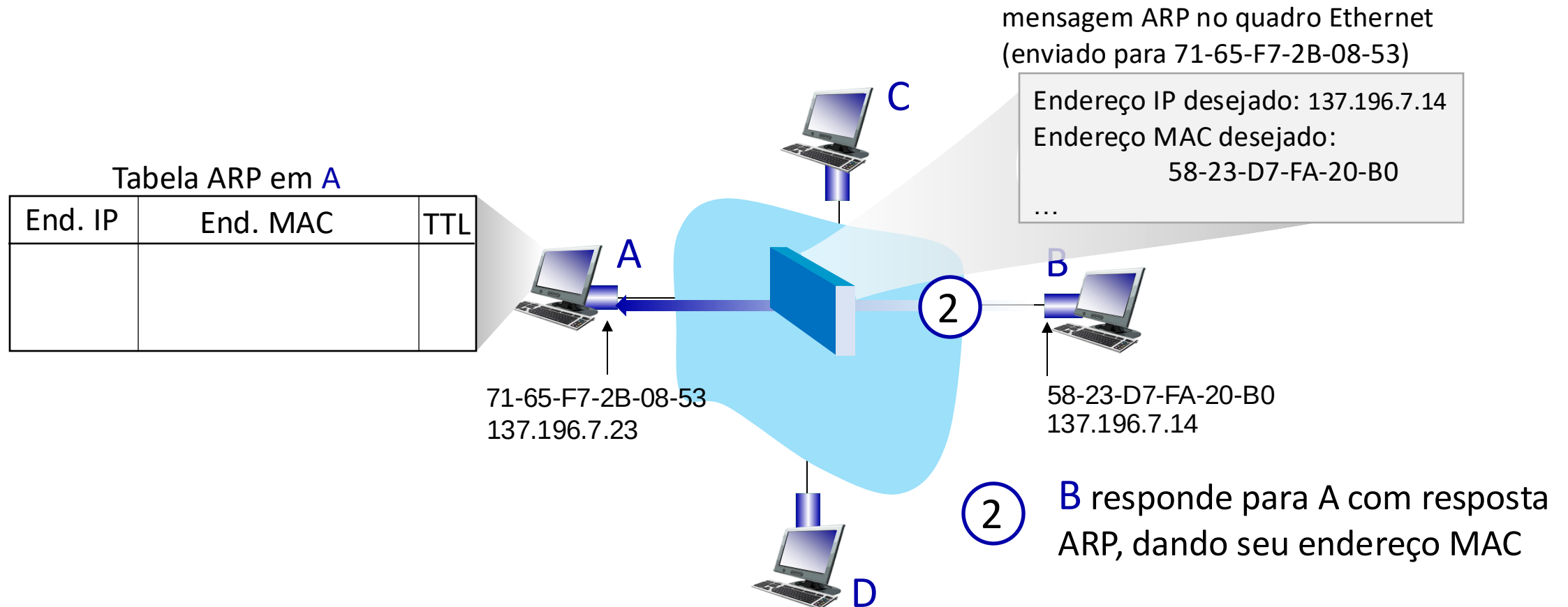
End. IP	End MAC	TTL



Protocolo ARP em ação

exemplo: A deseja enviar datagrama para B

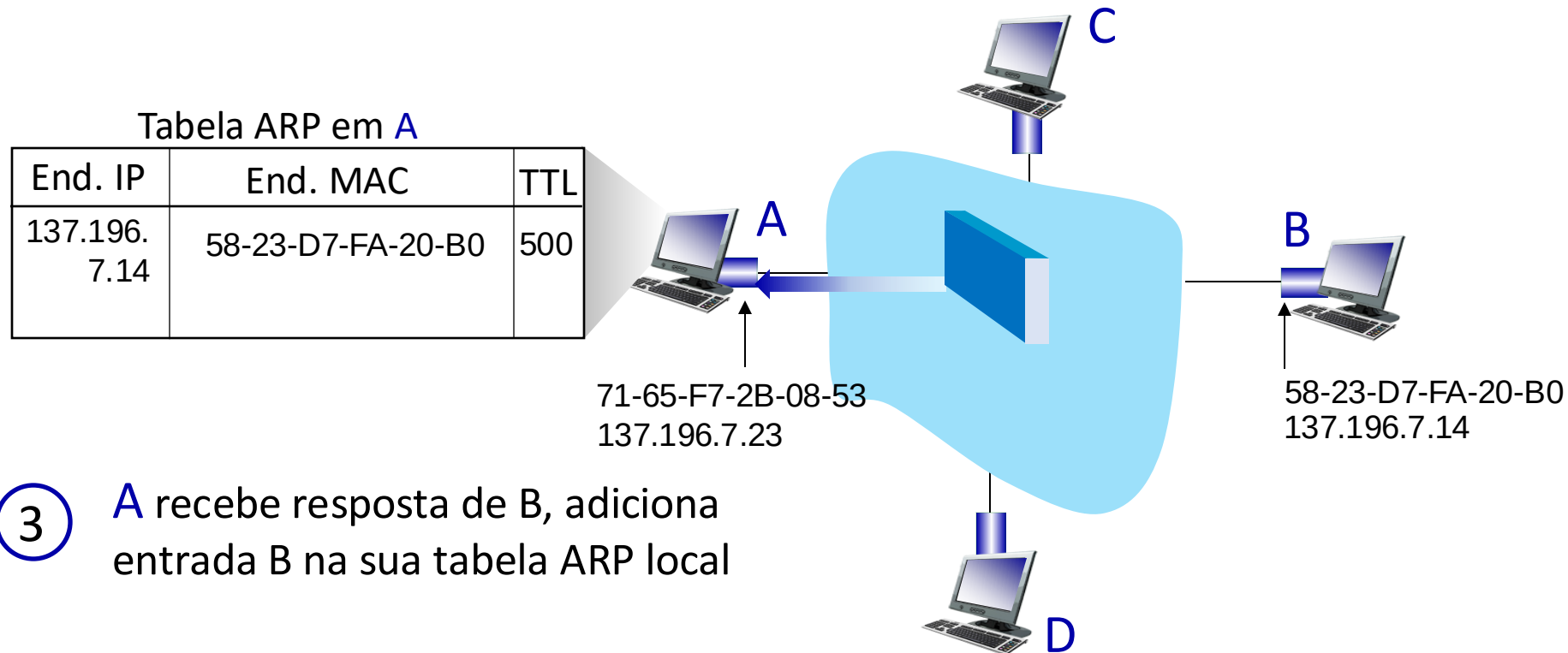
- endereço MAC de B não está na tabela ARP de A, então A usa ARP para obter o endereço MAC de B



Protocolo ARP em ação

exemplo: A deseja enviar datagrama para B

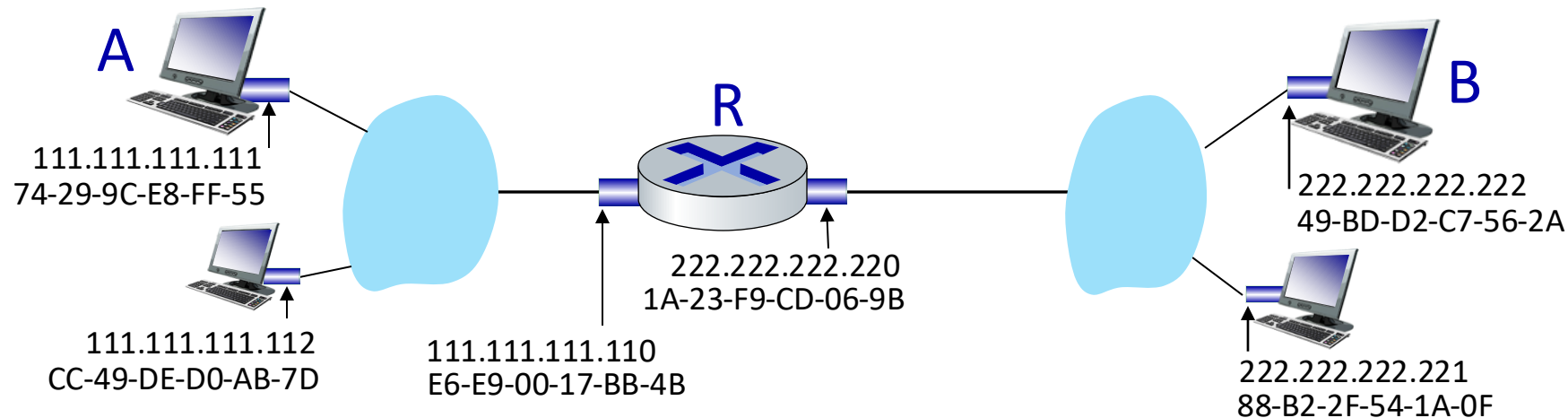
- endereço MAC de B não está na tabela ARP de A, então A usa ARP para obter o endereço MAC de B



Endereçamento: repassando para outra subrede

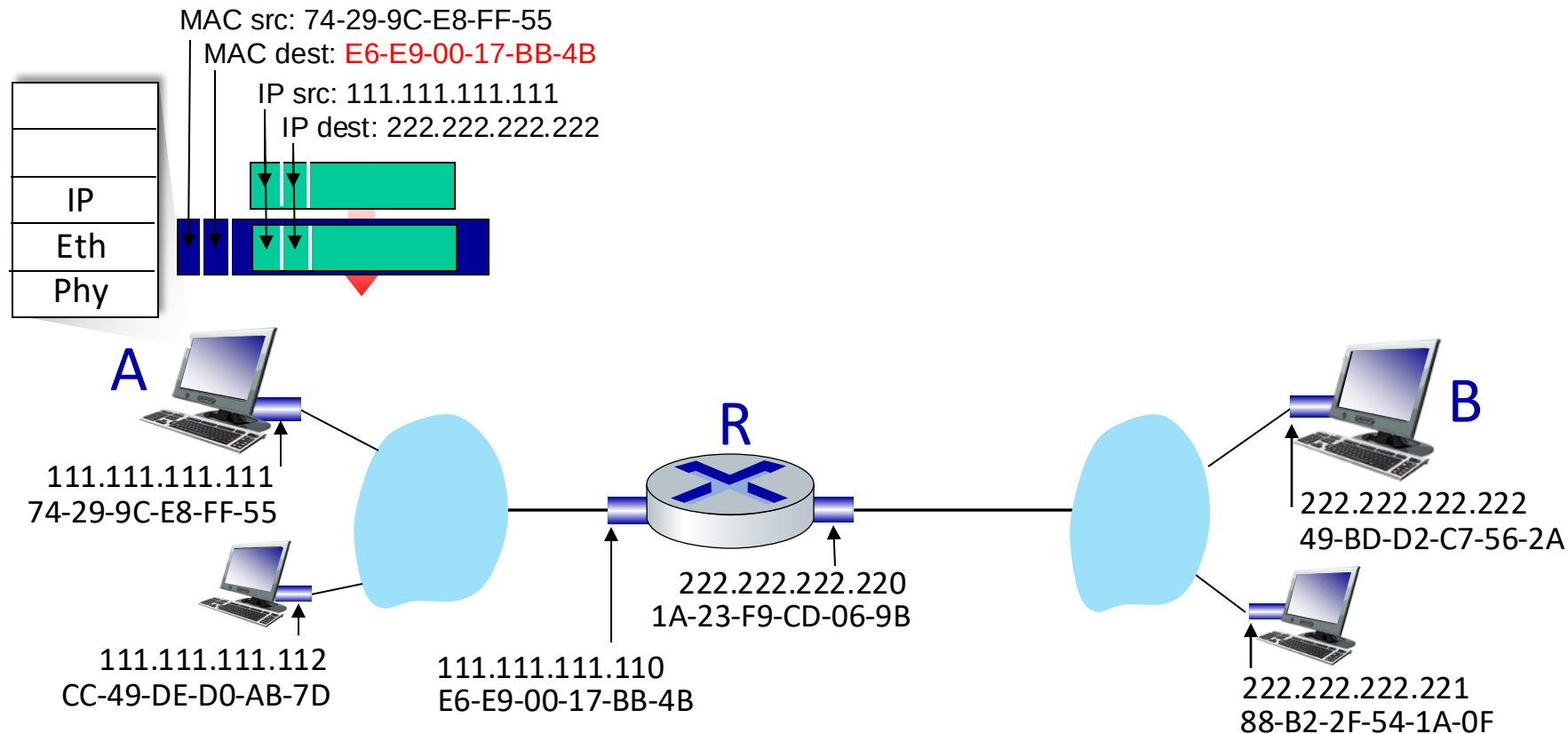
encaminhamento: **enviando um datagrama de A para B via R**

- foco no endereçamento – nas camadas IP (datagrama) e MAC (quadro)
- assuma que:
 - A conhece o endereço IP de B
 - A conhece o endereço IP do próximo roteador, R (como?)
 - A conhece o endereço MAC de R (como?)



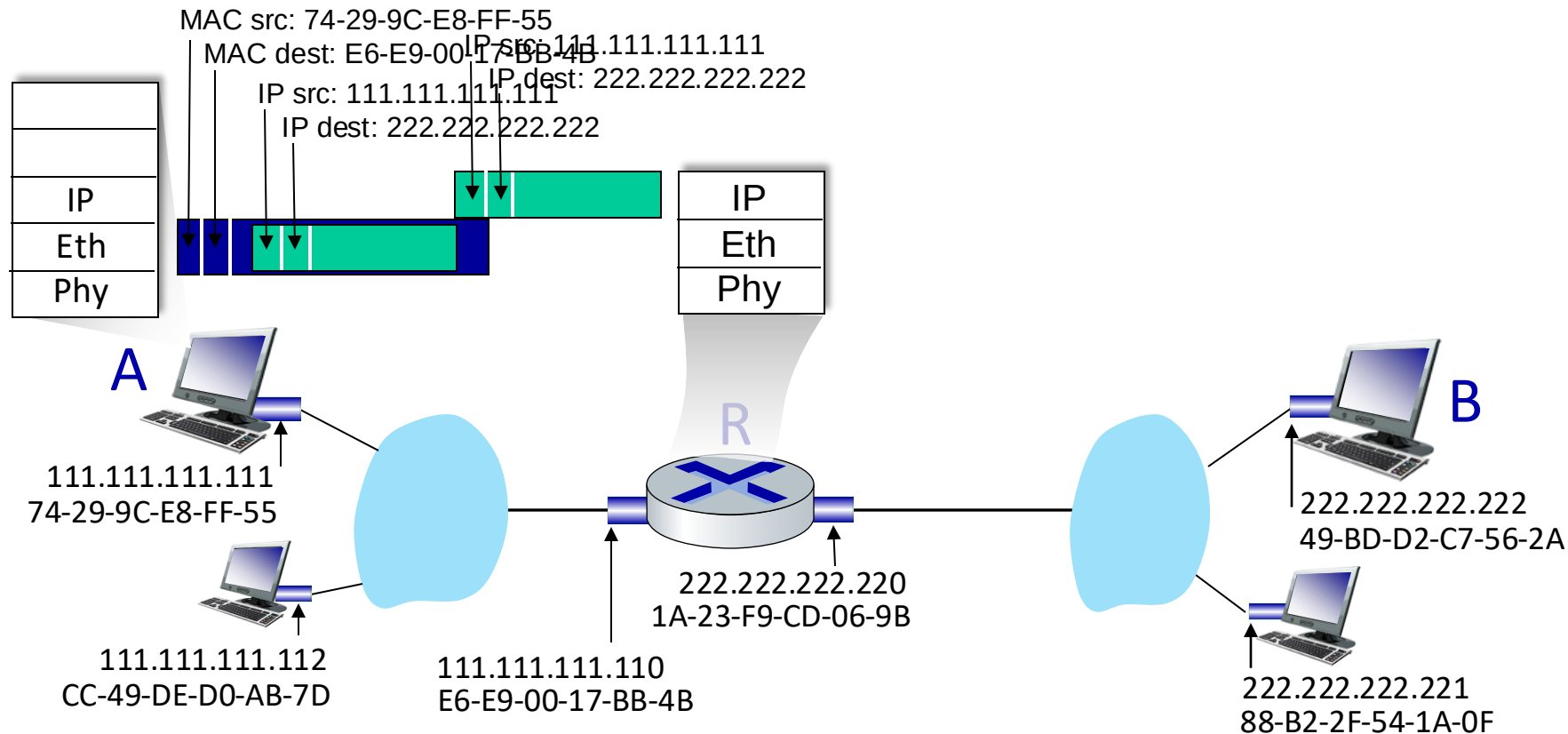
Endereçamento: repassando para outra subrede

- A cria datagrama IP com IP de origem A, destino B
- A cria quadro da camada de enlace contendo o datagrama IP de A-para-B
 - o destino do quadro é o endereço MAC de **R**



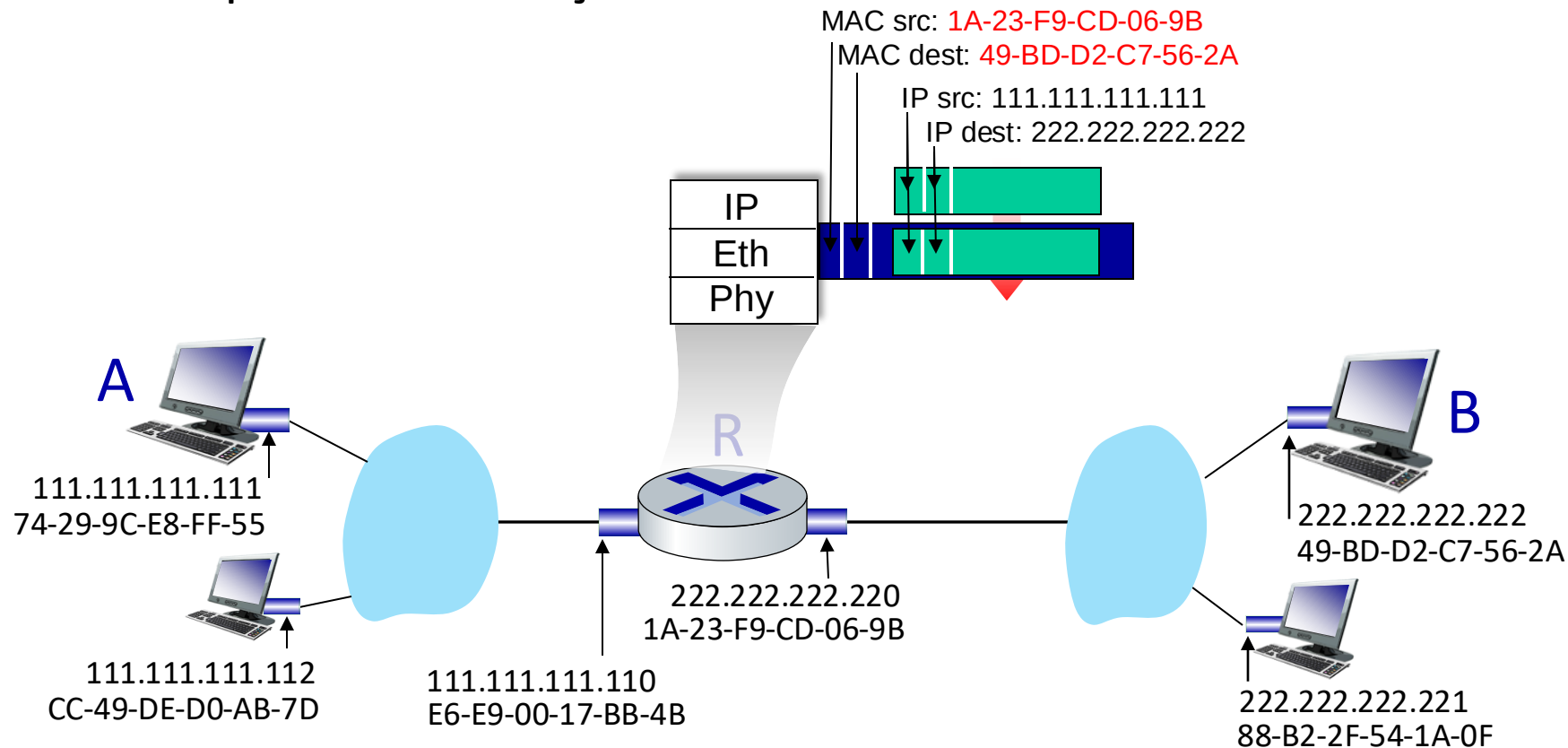
Endereçamento: repassando para outra subrede

- quadro enviado de A para R
- quadro recebido em R, datagrama removido, passado para o IP



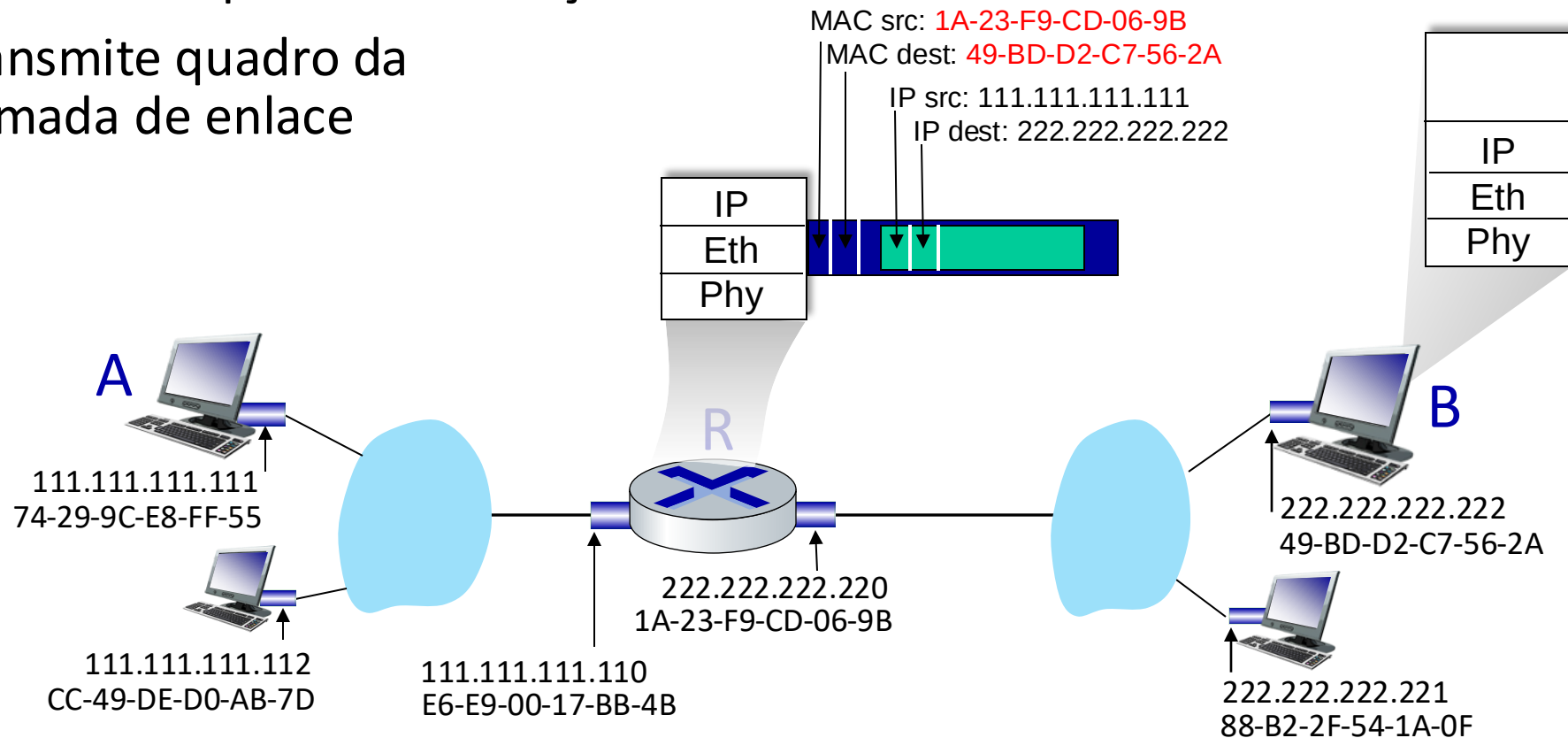
Endereçamento: repassando para outra subrede

- R obtém a interface de saída, passa o datagrama com IP origem A, destino B para a camada de enlace
- R cria quadro da camada de enlace contendo o datagrama IP de A-para-B. Endereço destino do quadro: endereço MAC de B



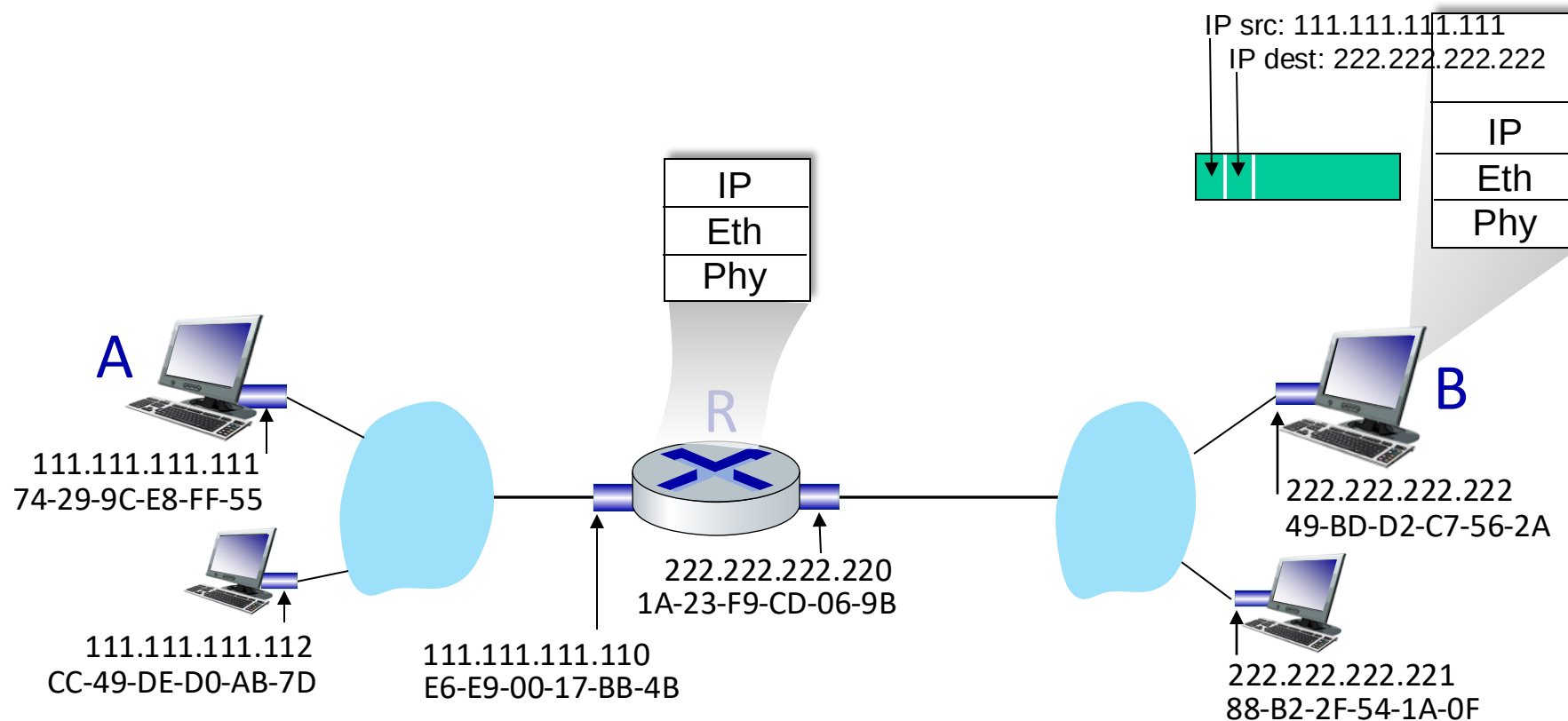
Endereçamento: repassando para outra subrede

- R obtém a interface de saída, passa o datagrama com IP origem A, destino B para a camada de enlace
- R cria quadro da camada de enlace contendo o datagrama IP de A-para-B. Endereço destino do quadro: endereço MAC de B
- transmite quadro da camada de enlace



Endereçamento: repassando para outra subrede

- B recebe o quadro, extrai o datagrama IP destinado a B
- B passa o datagrama para cima, para o IP



Camada de Enlace, LANs: roteiro

- introdução
- detecção, correção de erros
- protocolos de acesso múltiplo
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter



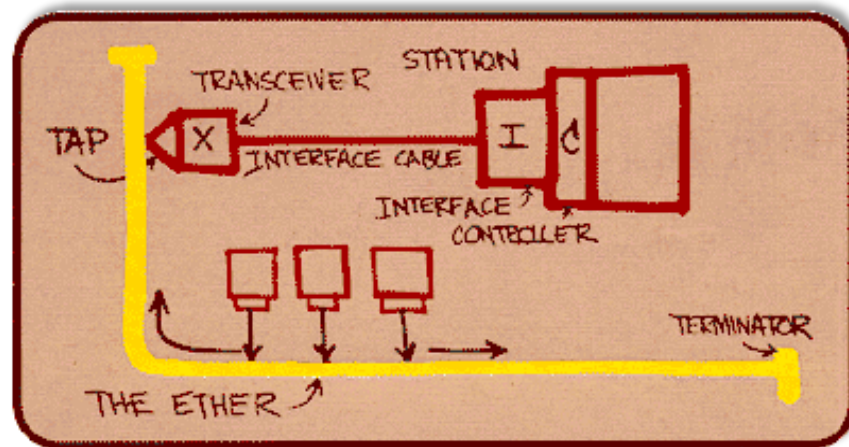
- um dia na vida de uma solicitação web

Ethernet

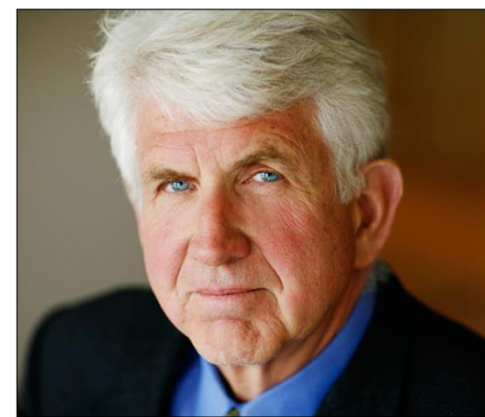
tecnologia “dominante” para LANs cabeadas:

- primeira tecnologia LAN usada em larga escala
- simples, barata
- acompanhou o aumento de velocidade: 10 Mbps – 400 Gbps
- chip único, múltiplas velocidades (e.g., Broadcom BCM5761)

*Rascunho de Metcalfe
sobre o Ethernet*



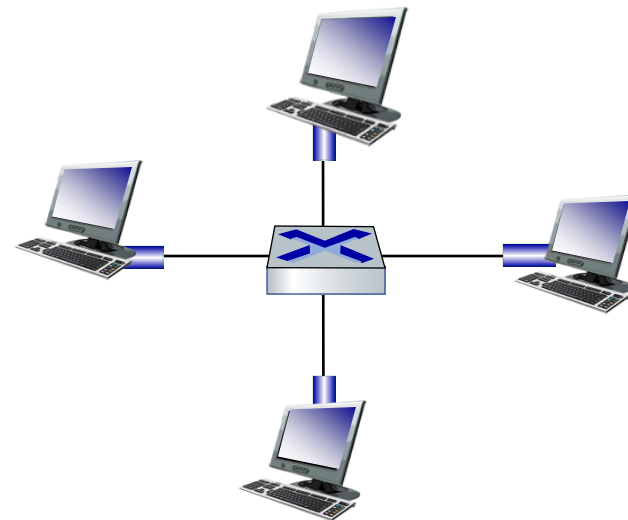
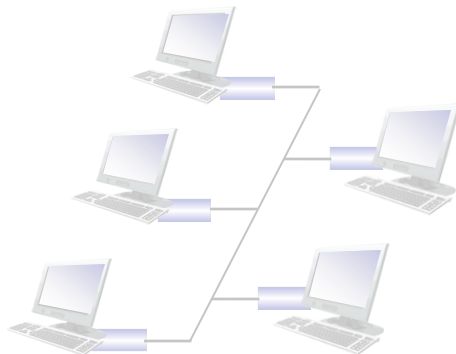
Bob Metcalfe: co-inventor do Ethernet,
Recebeu o 2022 ACM Turing Award



Ethernet: topologia física

- **barramento:** popular até meados dos anos 90
 - todos os nós no mesmo domínio de colisão (podem colidir um com o outro)
- **Comutada (*switched*):** prevalente hoje
 - *switch* ativo de camada de enlace no centro
 - cada “ramo” roda um protocolo Ethernet (separado): os nós não colidem uns com os outros

barramento:
cabo coaxial



comutada

Estrutura do quadro Ethernet

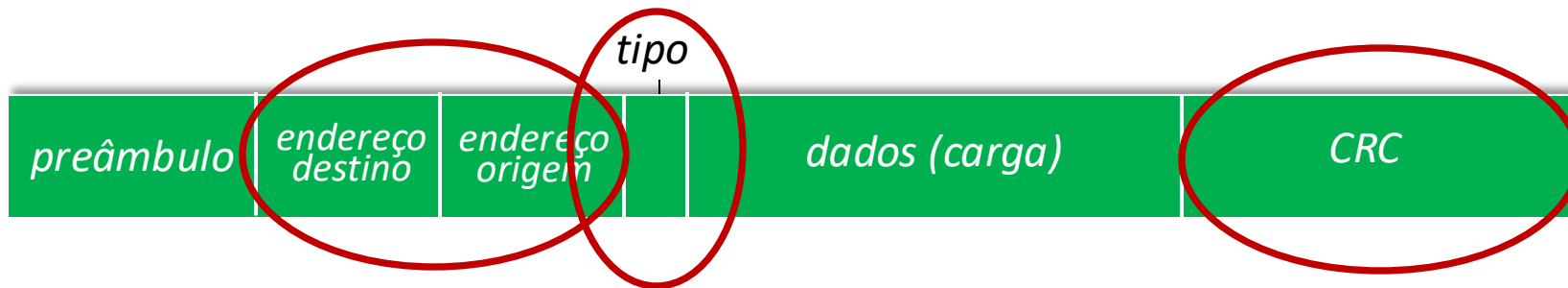
interface transmissora encapsula o datagrama IP (ou pacote de outro protocolo de camada de rede) no **quadro Ethernet**



preâmbulo:

- usado para sincronizar os relógios do receptor, transmissor
- 7 bytes com o padrão 10101010 seguido por um byte com o padrão 10101011

Estrutura do quadro Ethernet (mais)



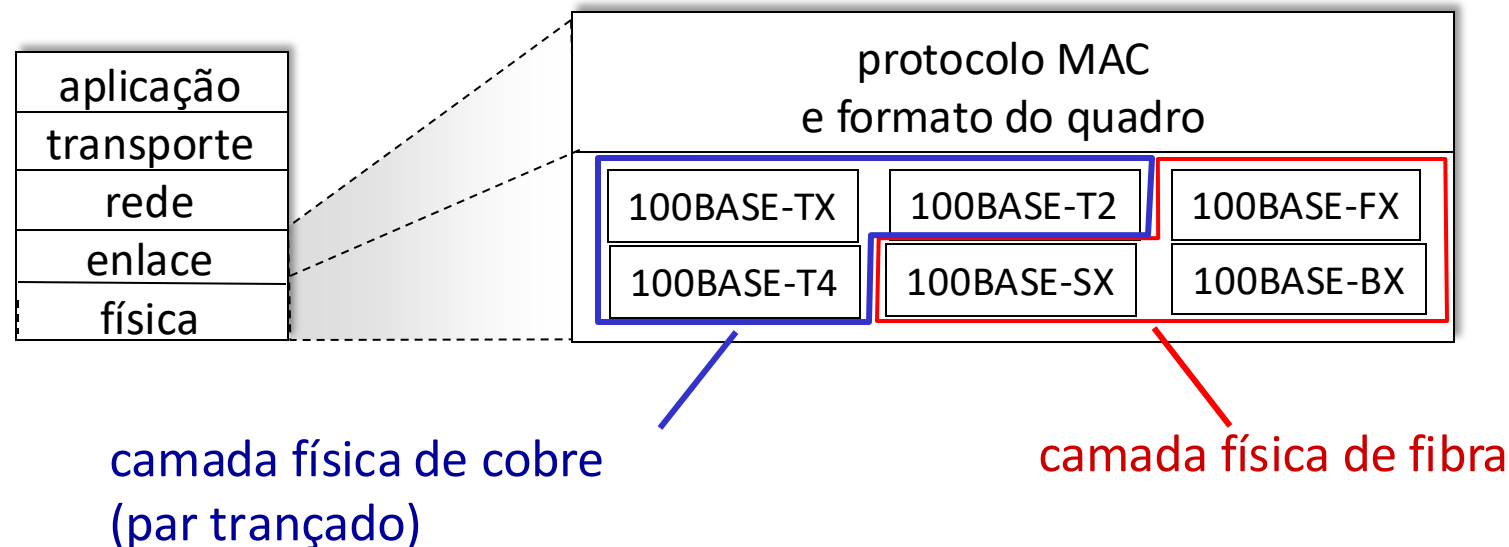
- **endereços:** 6 bytes para endereços MAC origem, destino
 - se o adaptador recebe um quadro com endereço destino igual ao seu, ou com endereço de *broadcast* (ex., pacote ARP), ele passa os dados do quadro para o protocolo da camada de rede
 - caso contrário, o adaptador descarta o quadro
- **tipo:** indica o protocolo da camada superior
 - usualmente o IP, mas há outras opções, ex., Novell IPX, AppleTalk
 - usado para demultiplexar no receptor
- **CRC:** verificação de redundância cíclica no receptor
 - erro detectado: quadro é descartado

Ethernet: não confiável, sem conexão

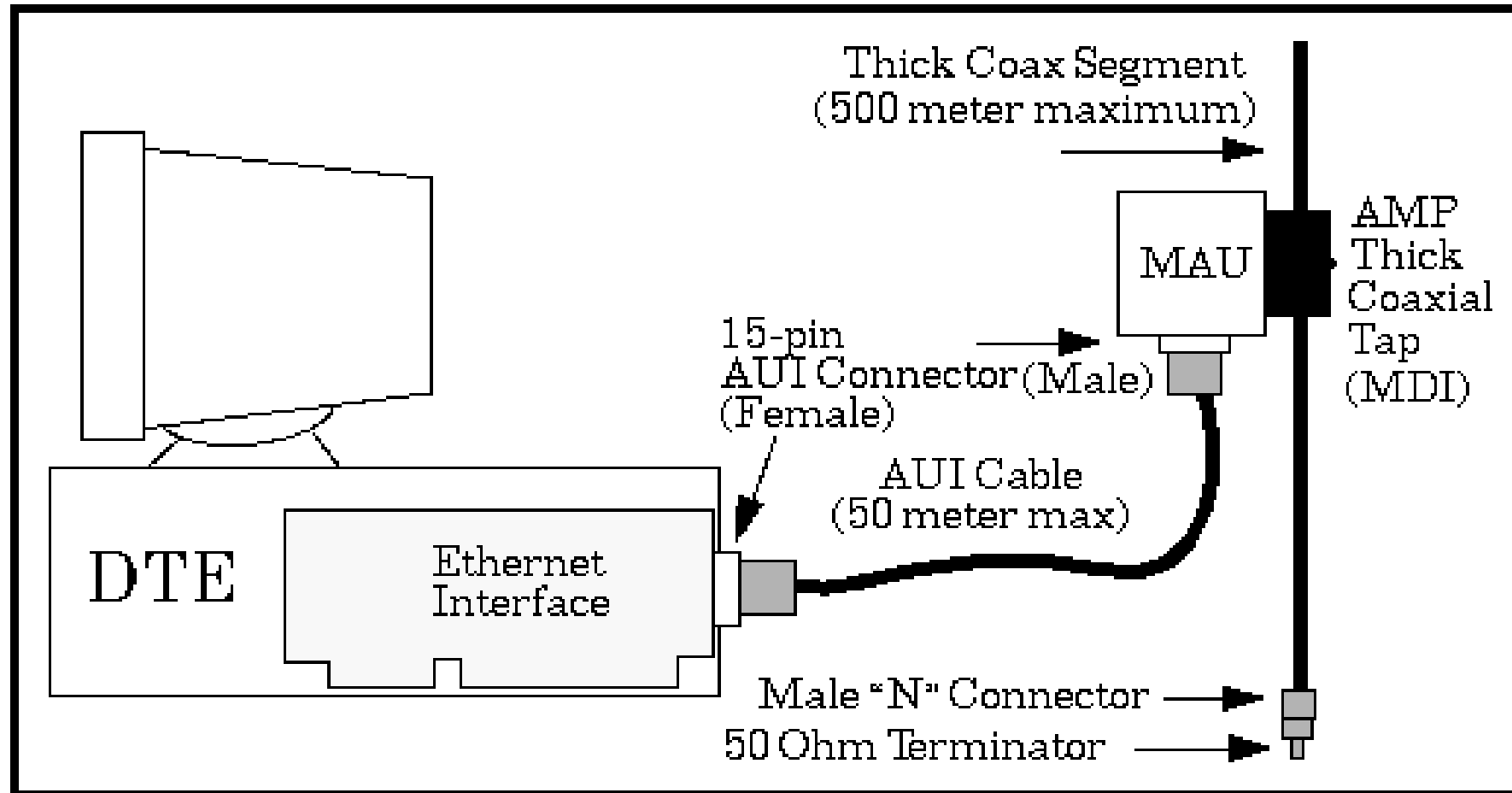
- **sem conexão:** não há saudação entre os NICs transmissor e receptor
- **não confiável:** NIC receptor não envia ACKs ou NAKs para o NIC transmissor
 - dados em quadros descartados são recuperados apenas se o transmissor usar algum protocolo confiável em camadas superiores (ex. TCP), caso contrário os dados descartados são perdidos
- protocolo MAC do Ethernet: **CSMA/CD** sem slots **com retirada binária**

Padrões Ethernet 802.3: camadas de enlace e física

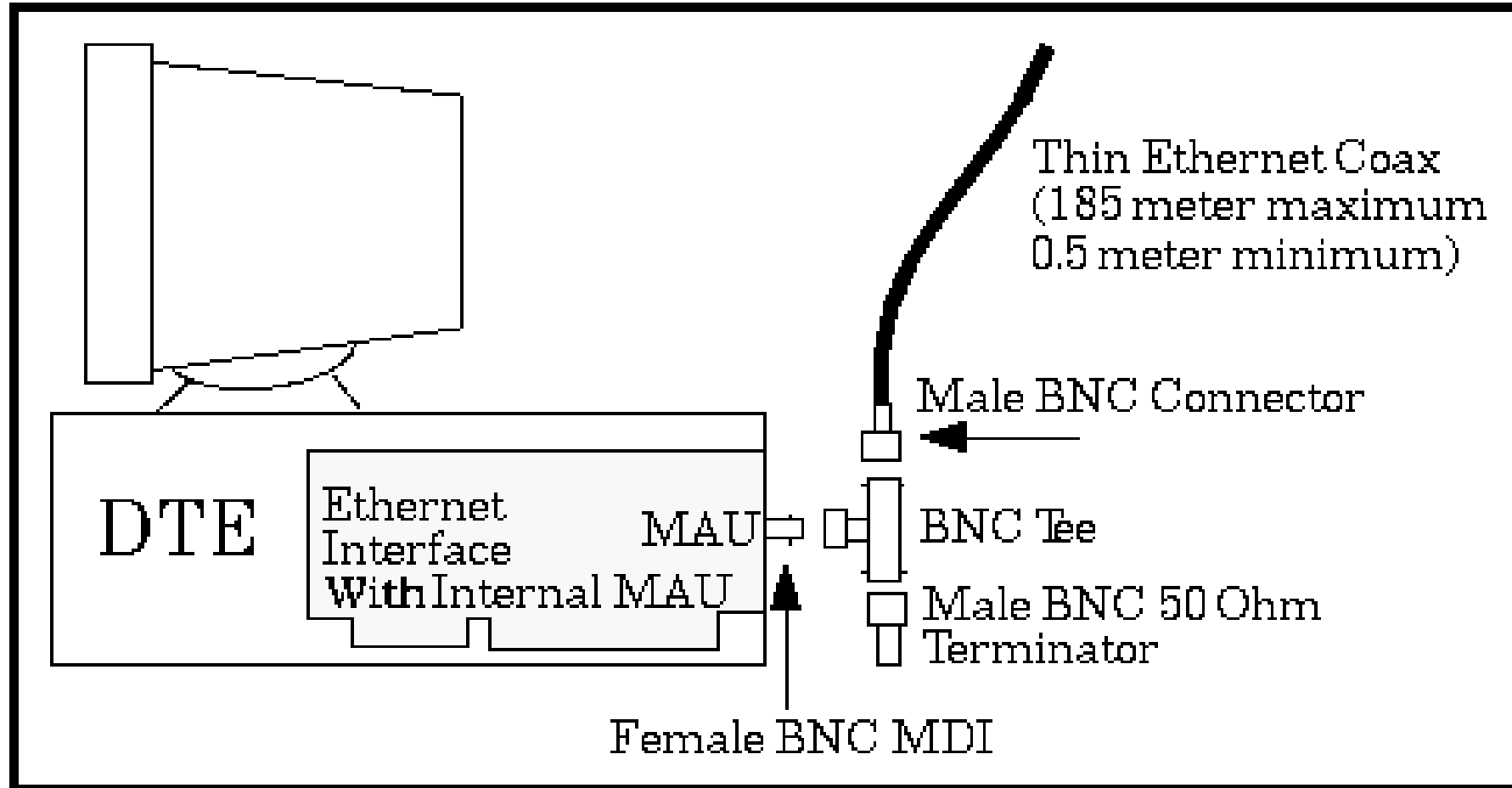
- *muitos* padrões Ethernet diferentes
 - mesmo protocolo MAC e formato do quadro
 - diferentes velocidades: 2 Mbps, 10 Mbps, 100 Mbps, 1Gbps, 10 Gbps, 40 Gbps
 - diferentes meios da camada física: fibra, cabo



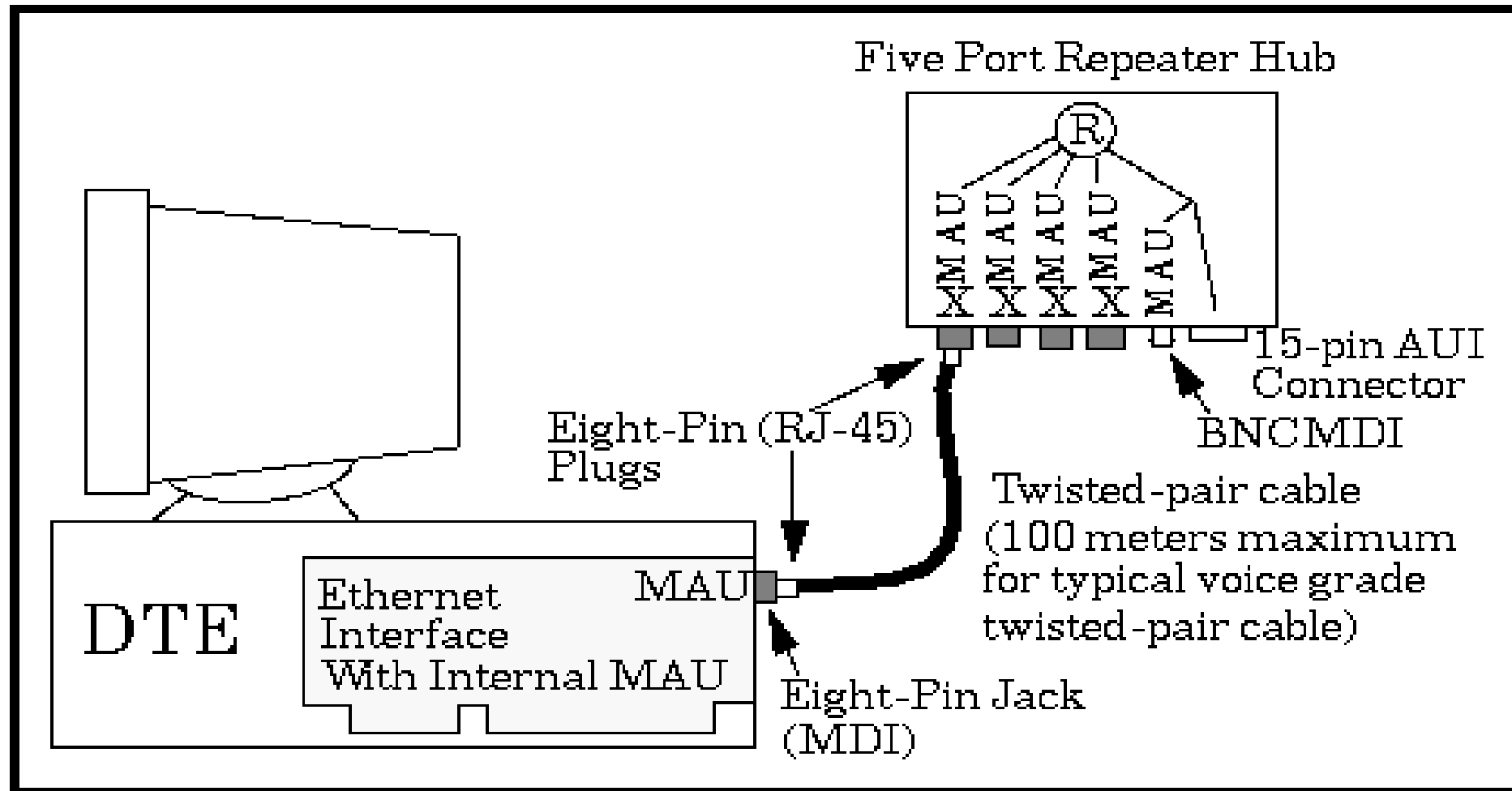
Evolução do Ethernet: 10Base5 (anos 70-80)



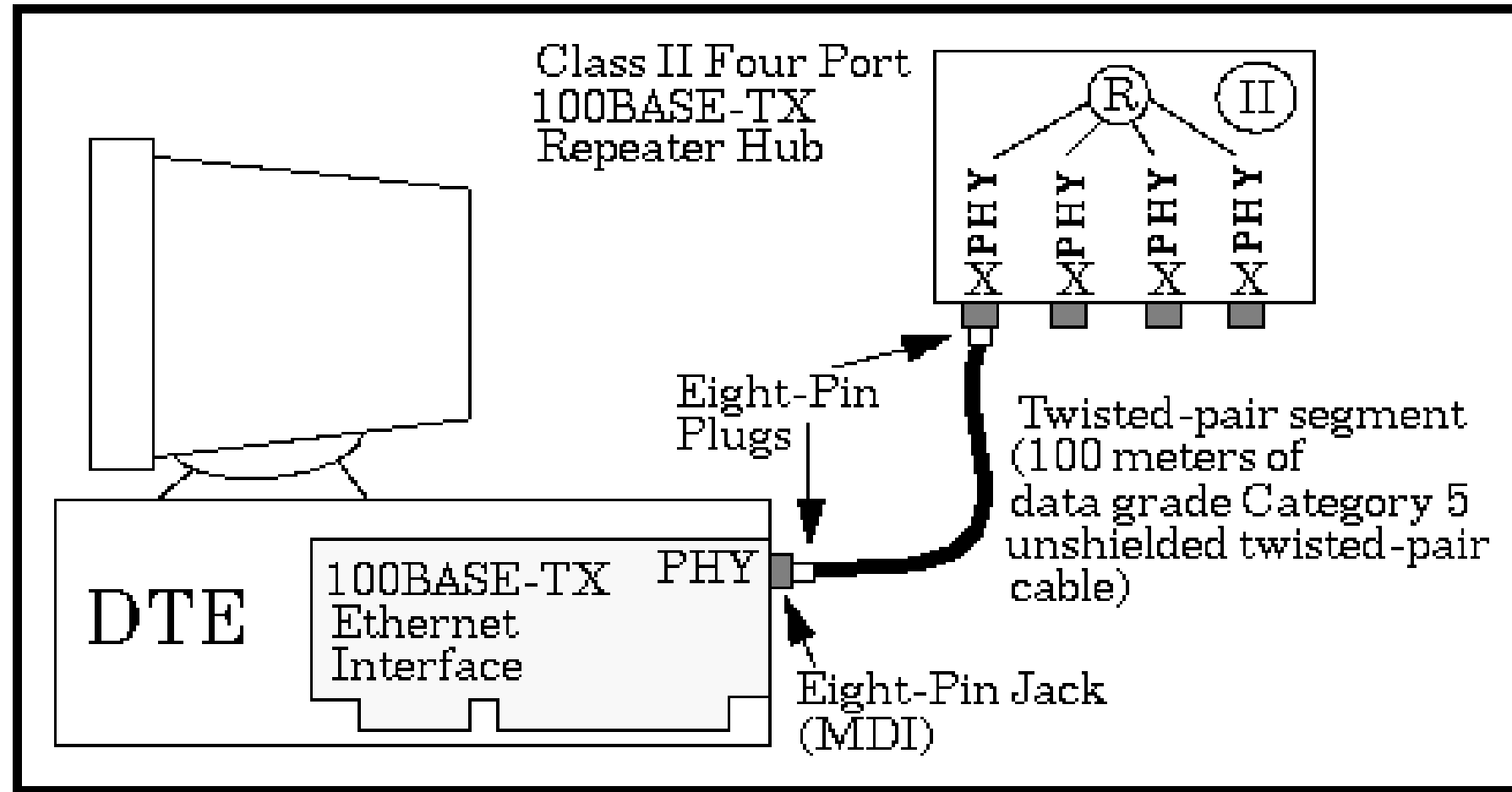
Evolução do Ethernet: 10Base2 (anos 80-90)



Evolução do Ethernet: 10Base-T (anos 90-00)



Evolução do Ethernet: 100Base-TX (anos 90-00)



Camada de Enlace, LANs: roteiro

- introdução
- detecção, correção de erros
- protocolos de acesso múltiplo
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter



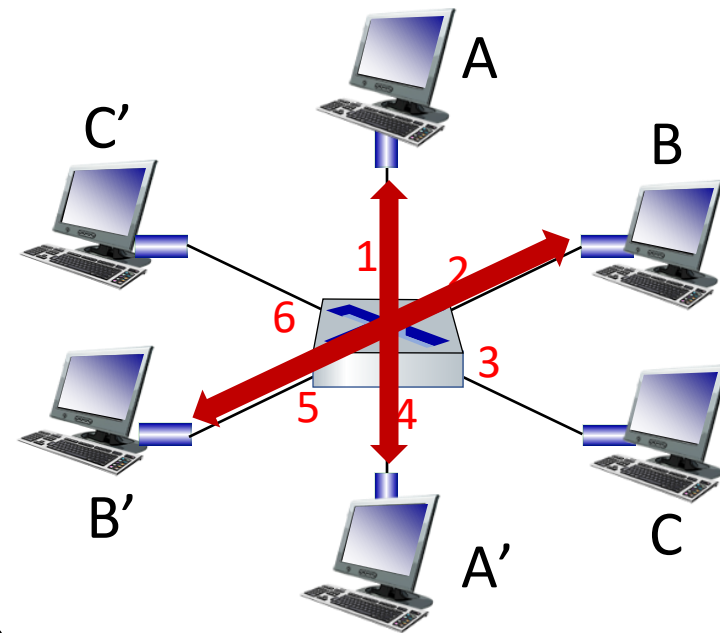
- um dia na vida de uma solicitação web

Comutador (*switch*) Ethernet

- O *switch* é um dispositivo da **camada de enlace**: tem um papel *ativo*
 - armazena, encaminha quadros Ethernet
 - examina o endereço MAC do quadro, repassa *seletivamente* o quadro para um ou mais enlaces de saída, ao repassar o quadro em um novo segmento, usa o CSMA/CD para acessá-lo
- **transparente**: hosts *desconhecem* a presença dos *switches*
- *plug-and-play, self-learning*
 - switches não precisam ser configurados (aprendem por si só)

Switch: múltiplas transmissões simultâneas

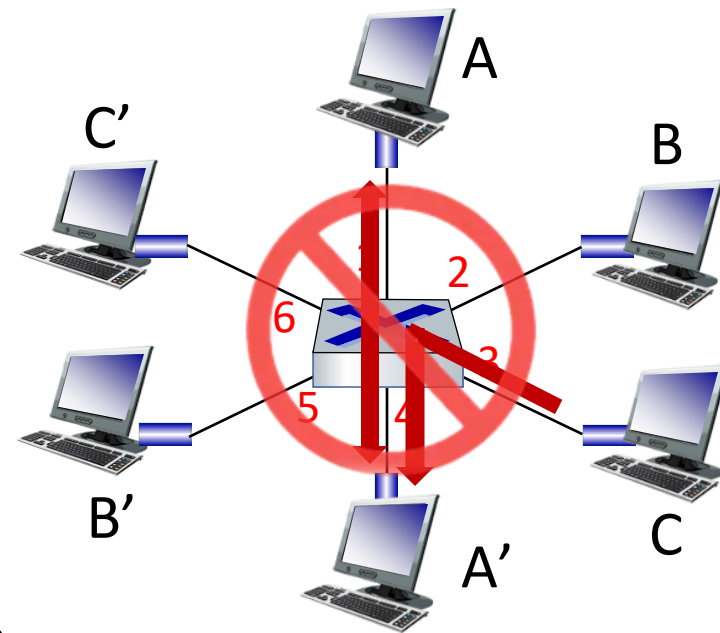
- hosts possuem conexões dedicadas, diretas para o *switch*
- *switches* armazenam quadros
- protocolo Ethernet é usado em cada enlace de entrada, então:
 - sem colisões; *full duplex*
 - cada enlace é o seu próprio domínio de colisão
- **comutação:** A-para-A' e B-para-B' podem transmitir simultaneamente, sem colisões



switch com seis interfaces (1,2,3,4,5,6)

Switch: múltiplas transmissões simultâneas

- hosts possuem conexões dedicadas, diretas para o *switch*
- *switches* armazenam quadros
- protocolo Ethernet é usado em cada enlace de entrada, então:
 - sem colisões; *full duplex*
 - cada enlace é o seu próprio domínio de colisão
- **comutação:** A-para-A' e B-para-B' podem transmitir simultaneamente, sem colisões
 - mas, A-para-A' e C-para-A' *não* pode ocorrer simultaneamente



switch com seis interfaces (1,2,3,4,5,6)

Tabela de repasse do *switch*

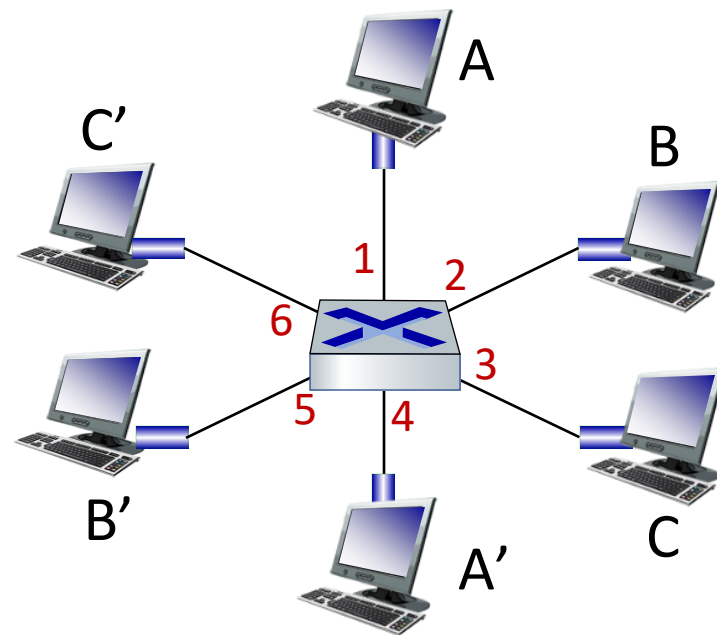
P: como o *switch* sabe que A' é alcançável através da interface 4, B' é alcançável através da interface 5?

R: cada *switch* possui uma **tabela de comutação**, cada entrada contém:

- (endereço MAC do host, interface para alcançar o host, carimbo de tempo)
- parece uma tabela de repasse!

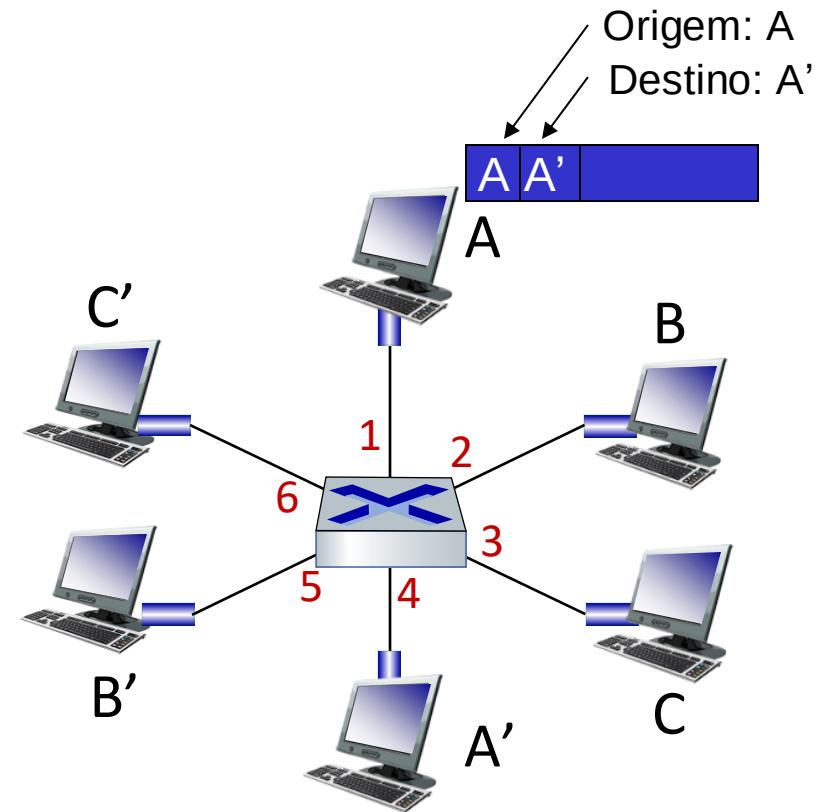
P: como as entradas são criadas, mantidas na tabela do *switch*?

- há algo como um protocolo de roteamento?



Switch: autoaprendizagem

- *switch* **aprende** quais hosts podem ser alcançados através de quais interfaces
 - quando o quadro é recebido, o *switch* aprende a localização do transmissor: segmento LAN de entrada
 - registra o par transmissor/localização na tabela de comutação



End. MAC	interface	TTL
A	1	60

Tabela de comutação (inicialmente vazia)

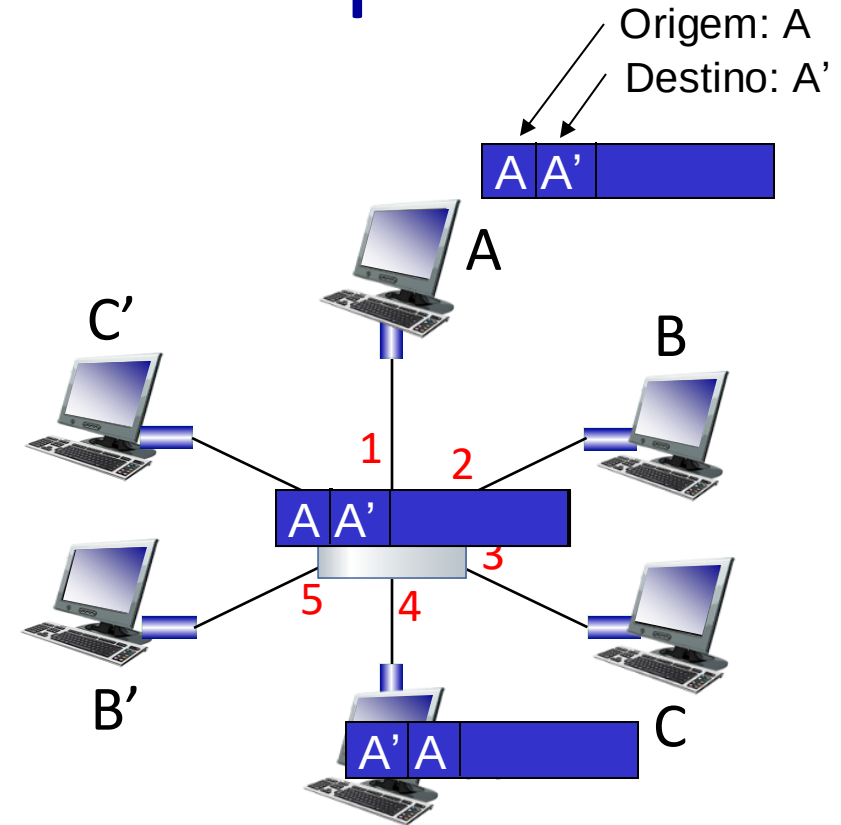
Switch: filtragem/repassse de quadros

quando um quadro é recebido num *switch*:

1. registra o enlace de entrada, endereço MAC do host transmissor
2. indexa a tabela de comutação usando o endereço MAC do destino
3. **se** encontrar uma entrada para o destino
então {
 se destino estiver no segmento de onde o quadro veio
 então descarta quadro
 caso contrário encaminha quadro na interface indicada pela entrada
}
 caso contrário inundação /* repassa para todas as interfaces exceto
para a interface em que foi recebido */

Exemplo de autoaprendizado e repasse

- destino do quadro, A',
localização desconhecida: **inundação**
- localização do destino A
conhecida:
**envio seletivo em
apenas um enlace**

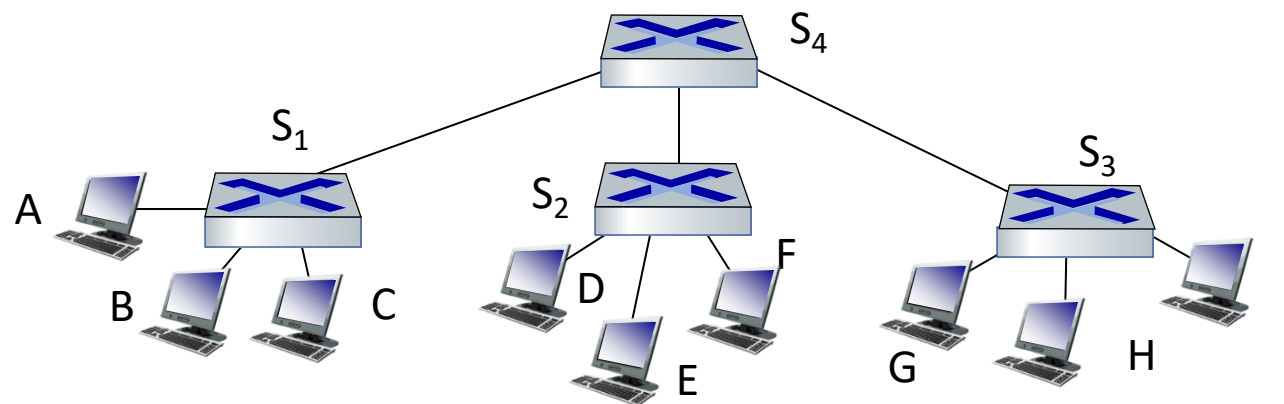


End. MAC	interface	TTL
A	1	60
A'	4	60

*tabela de
comutação
(inicialmente
vazia)*

Interconexão de *switches*

switches autodidatas podem ser interligados:

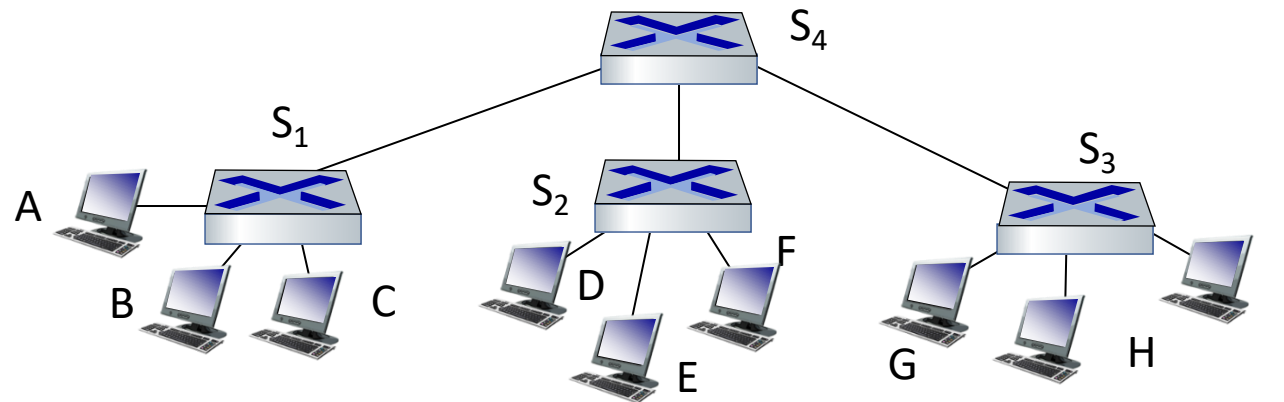


P: ao enviar de A para G – como S₁ sabe que deve repassar o quadro destinado a G por S₄ e S₃?

- R: autoaprendizado! (funciona exatamente da mesma forma que no caso de um único *switch*!)

Exemplo de múltiplos *switches* autodidatas

Suponha que C envia quadro para I, I responde para C



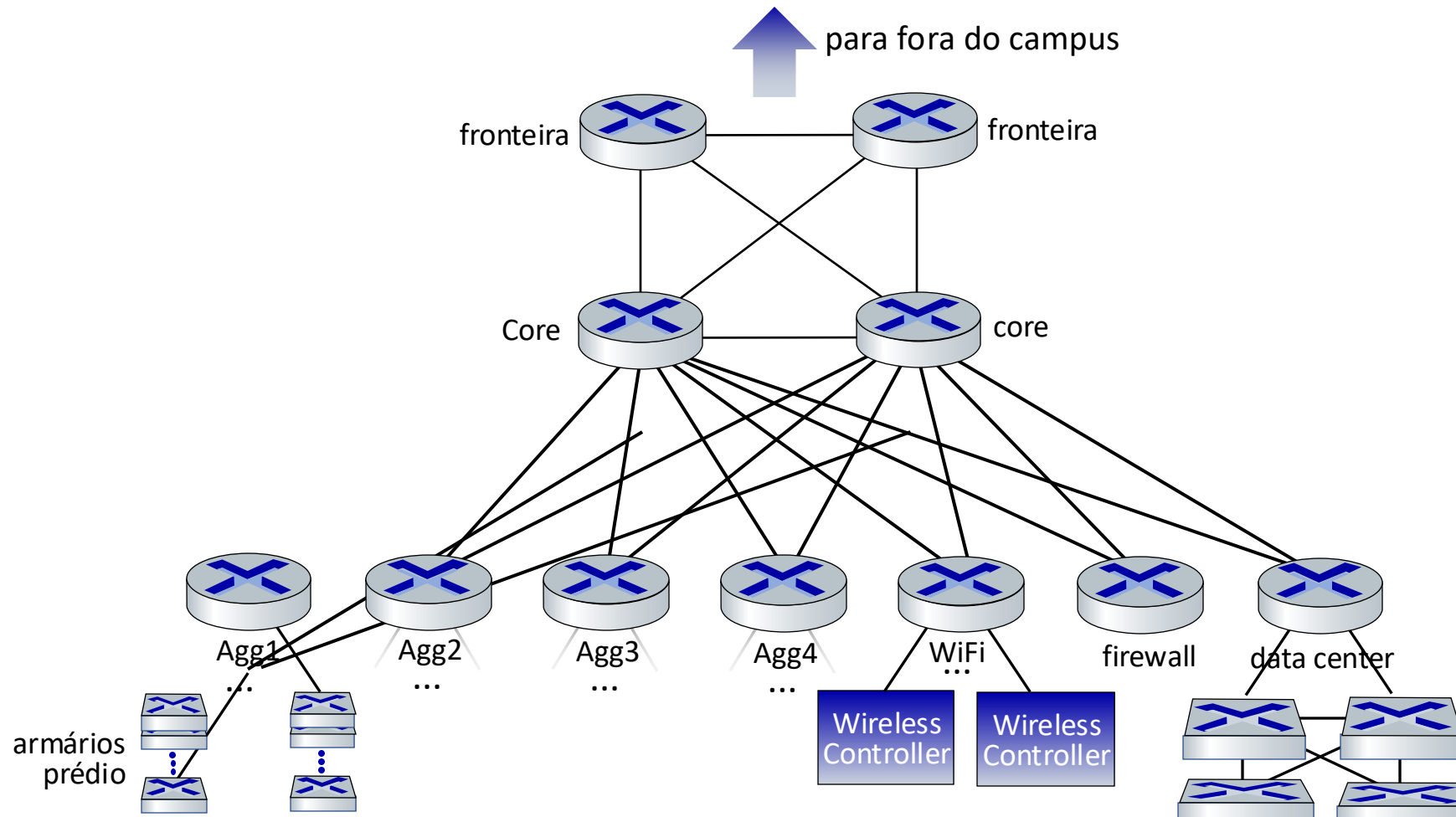
P: mostre as tabelas de comutação e repasse de pacotes em S_1 , S_2 , S_3 , S_4

Rede da UMass - Detalhes

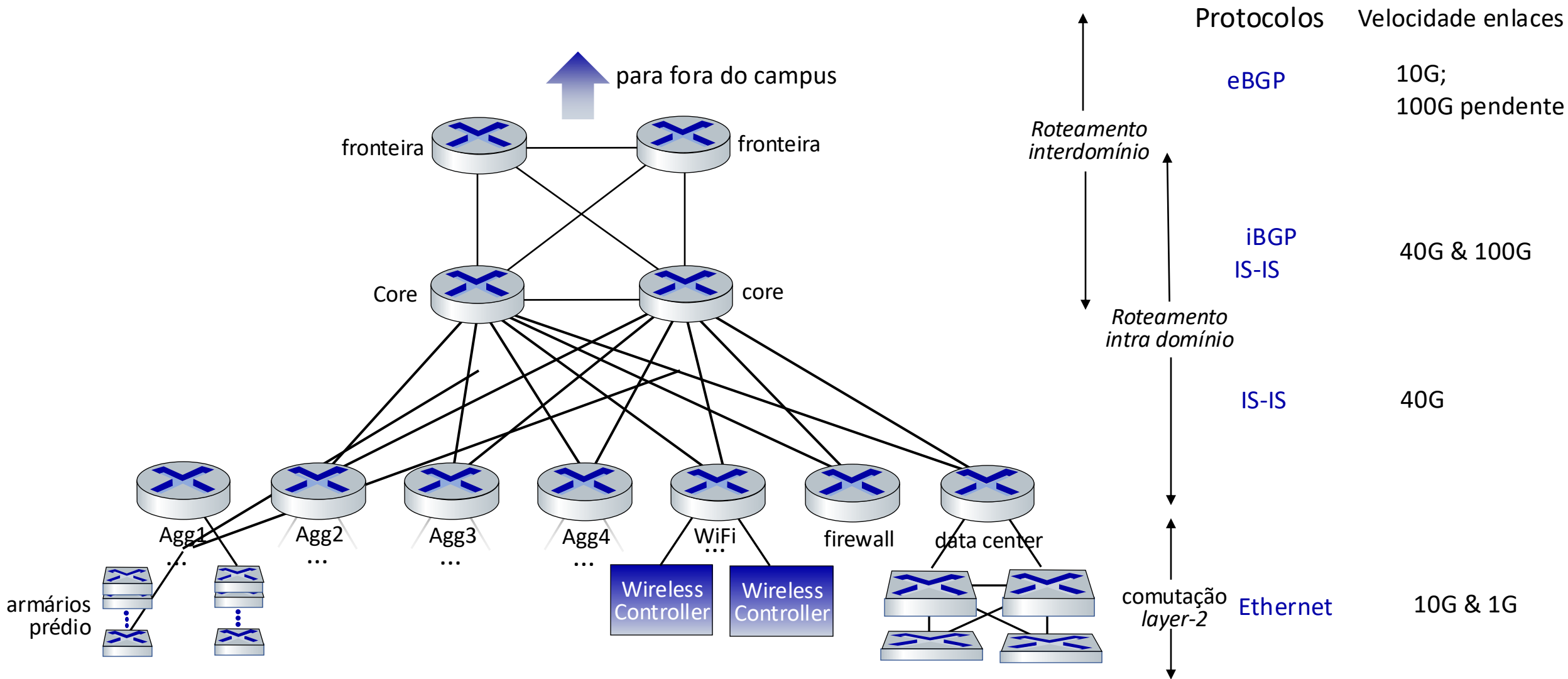
Rede da UMass:

- 4 firewalls
- 10 roteadores
- 2000+ switches de rede
- 6000 pontos de acesso sem fio
- 30.000 conectores ativos de rede
- 55.000 dispositivos sem-fio do usuário final

... instalado,
operado e
mantido por ~15
pessoas



Rede da UMass - Detalhes



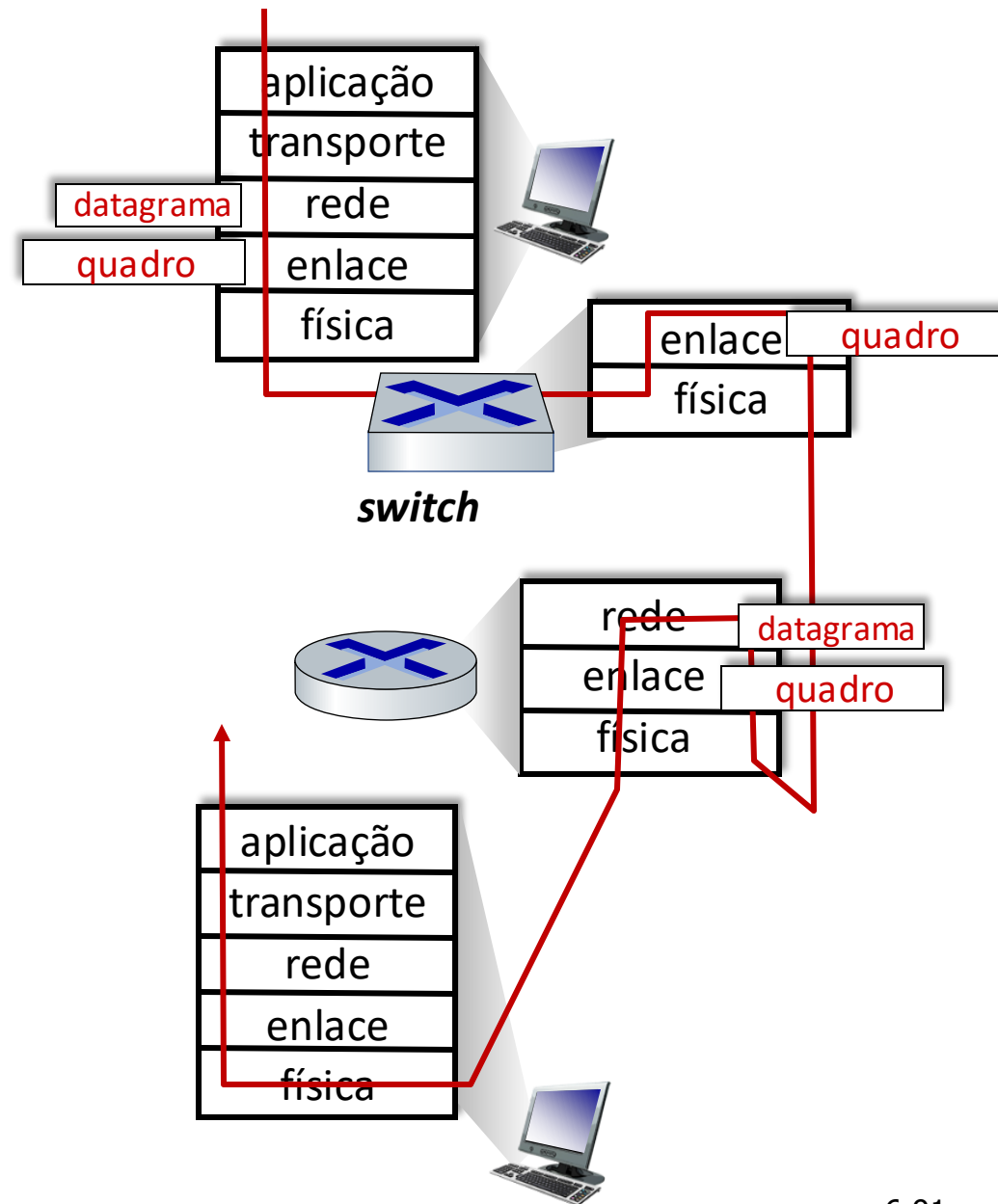
Switches vs. roteadores

ambos armazenam-e-repassam:

- **roteadores:** dispositivos da camada de rede (examinam os cabeçalhos da camada de rede)
- **switches:** dispositivos da camada de enlace (examinam os cabeçalhos da camada de enlace)

ambos possuem tabelas de repasse:

- **roteadores:** calculam tabelas usando algoritmos de roteamento, endereços IP
- **switches:** aprendem tabelas de repasse usando inundação, aprendizado, endereços MAC



Camada de Enlace, LANs: roteiro

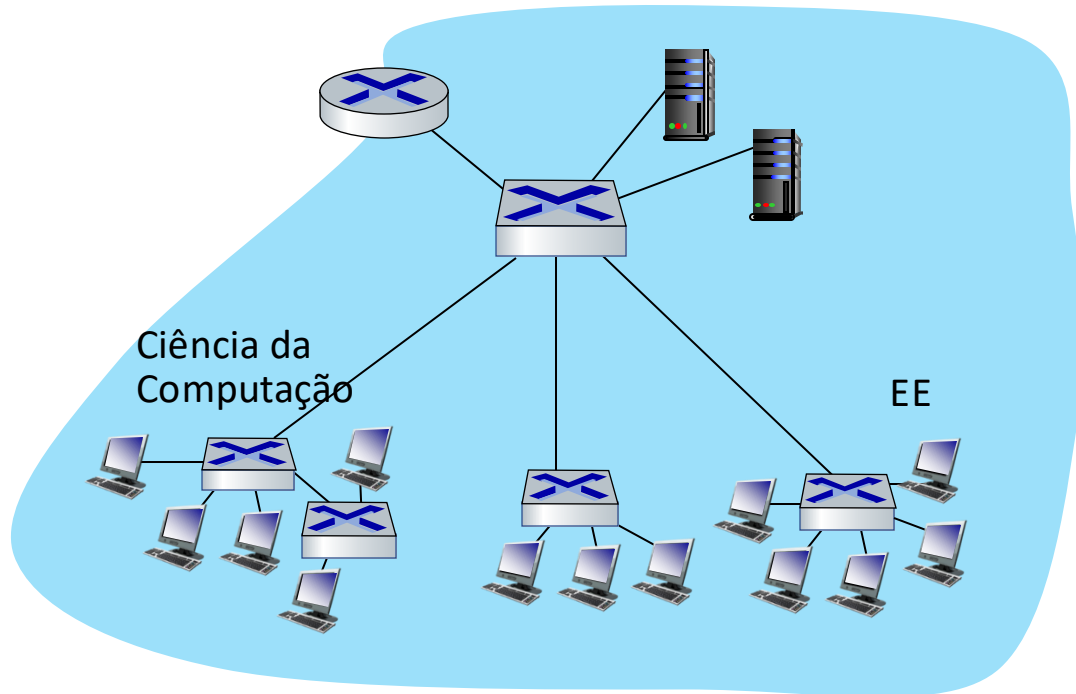
- introdução
- detecção, correção de erros
- protocolos de múltiplo acesso
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter



- um dia na vida de uma solicitação web

LANs Virtuais (VLANs): motivação

P: o que ocorre quando o tamanho da LAN cresce, usuários mudam de ponto de conexão?

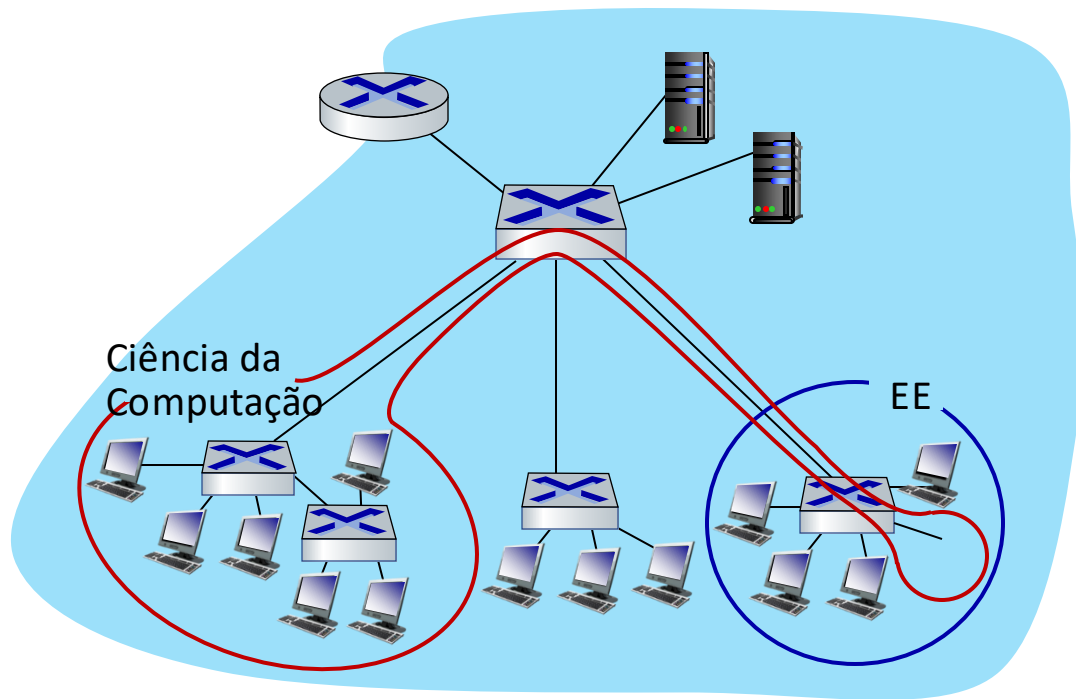


domínio único de difusão:

- *escala:* todo o tráfego de difusão de camada-2 (ARP, DHCP, MAC desconhecido) deve cruzar toda a LAN
- questões de eficiência, segurança, privacidade

LANs Virtuais (VLANs): motivação

Q: o que ocorre quando o tamanho da LAN cresce, usuários mudam de ponto de conexão?



domínio único de difusão:

- *escala*: todo o tráfego de difusão de camada-2 (ARP, DHCP, MAC desconhecido) deve cruzar toda a LAN
- questões de eficiência, segurança, privacidade

questões administrativas:

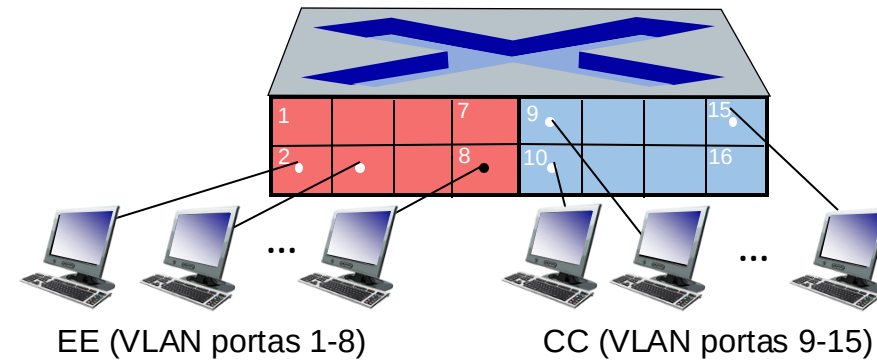
- usuário de CC muda escritório para EE - conectado *fisicamente* ao switch EE, mas deseja permanecer *logicamente* conectado ao switch CC

VLANs baseadas em Portas

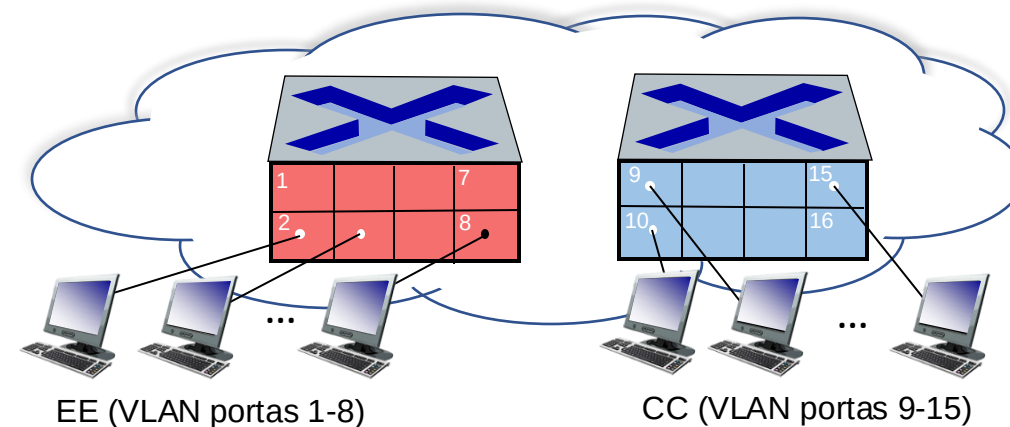
Virtual Local Area Network (VLAN)

capacitações de VLAN dos *switch(es)* podem ser configuradas para definir múltiplas LANs *virtuais* sobre uma única infraestrutura de LAN física.

VLAN baseada em porta: portas do switch agrupadas (pelo software de gerência do switch) de modo que *único* switch físico

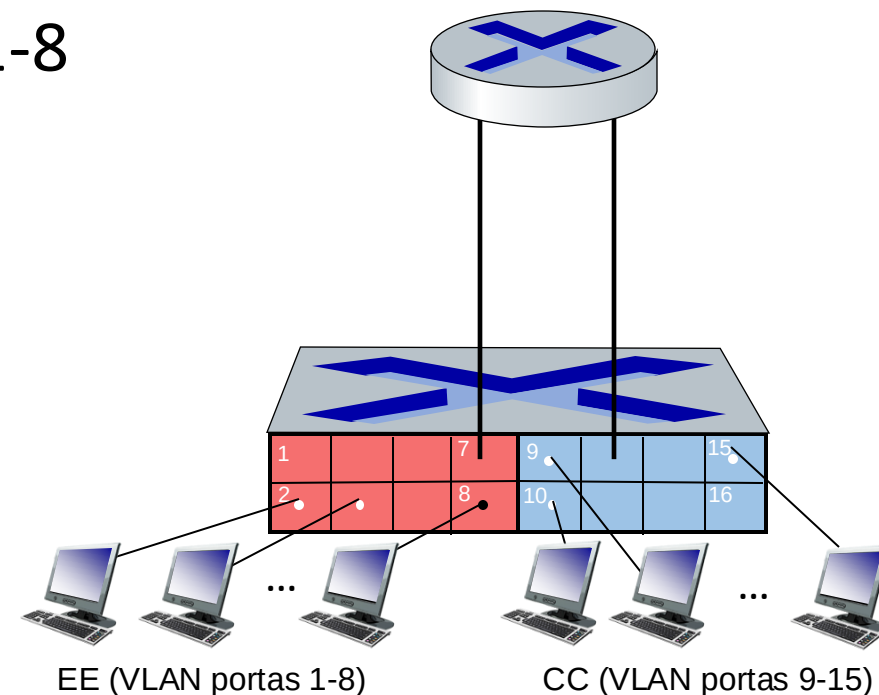


... opera como **múltiplos** switches virtuais

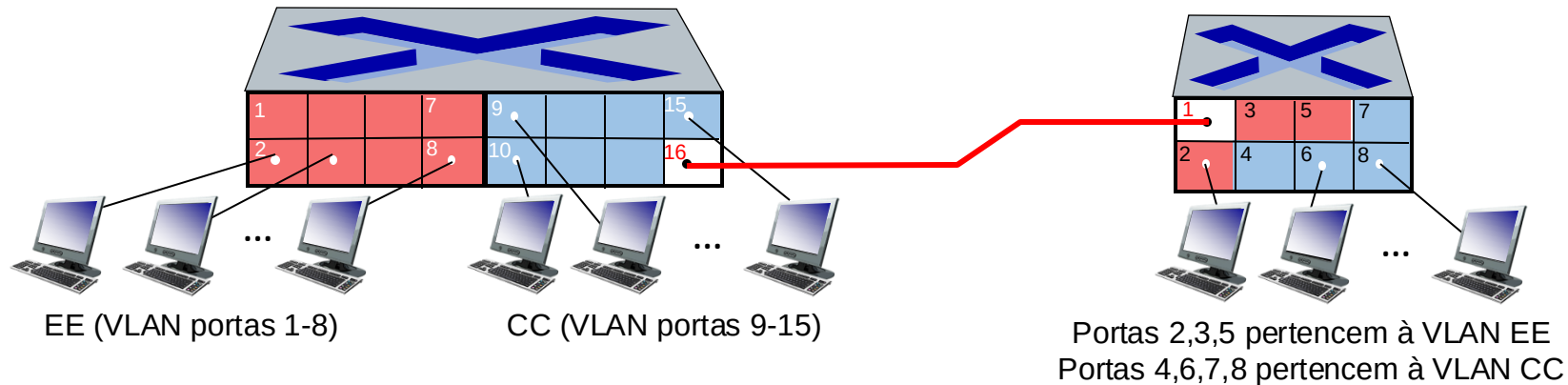


VLANs baseadas em Portas

- **isolamento de tráfego:** quadros para/das portas 1-8 podem *apenas* alcançar portas 1-8
 - pode-se definir VLAN baseada em endereços MAC dos pontos finais ao invés da porta do switch
- **filiação dinâmica:** portas podem ser alocadas dinamicamente entre VLANs
- **repasse entre VLANs:** realizada via roteador (exatamente como para *switches* separados)
 - na prática os fabricantes vendem *switches* combinados com roteadores



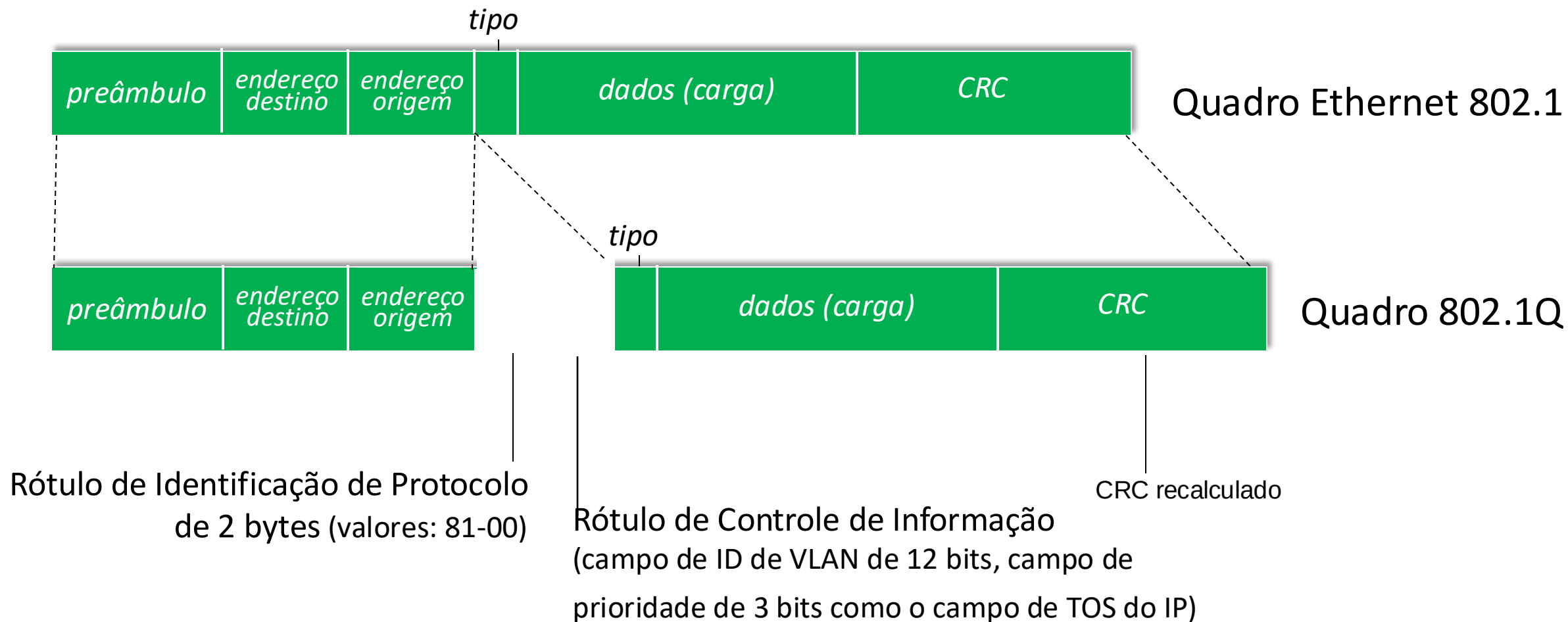
VLANs formada por múltiplos *switches*



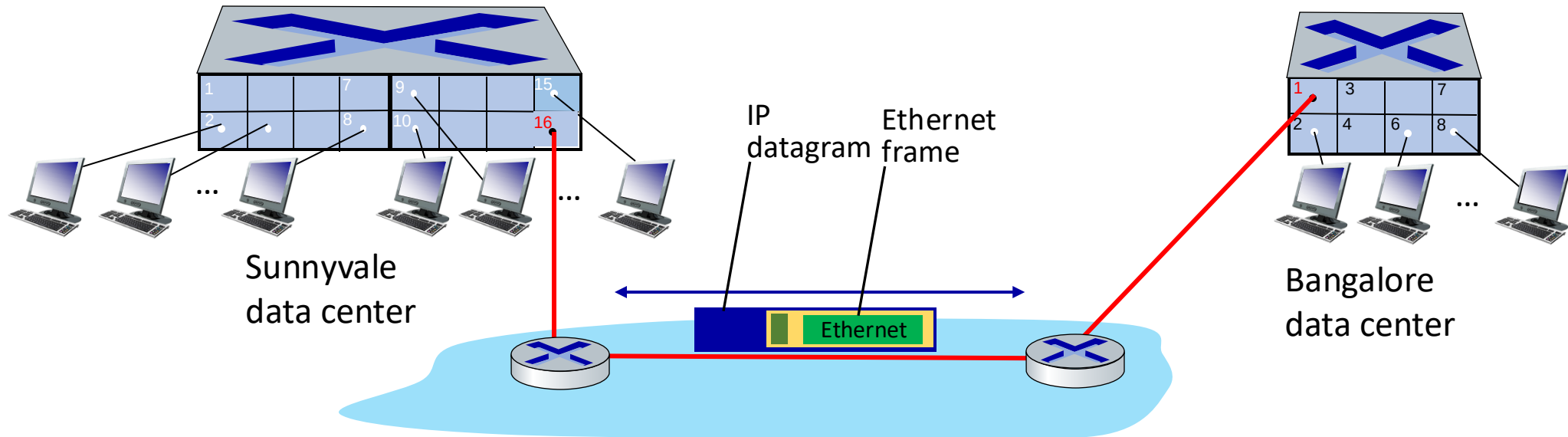
porta tronco: transporta quadros entre VLANs definidas sobre múltiplos switches físicos

- quadros repassados dentro de um VLAN entre diferentes switches não podem ser quadros 802.1 básicos (precisam transportar informação de ID da VLAN)
- protocolo 802.1q adiciona/remove campos adicionais de cabeçalho para quadros repassados entre portas tronco

Formato do quadro VLAN 802.1Q



EVPN: Ethernet VPNs (aka VXLANs)



Switches Ethernet Layer-2 conectados *logicamente* um ao outro (e.g., usando IP como uma rede *abaixo*)

- Quadros Ethernet transportados *dentro* de datagramas IP entre sites
- “esquema de *tunelamento* para *redes sobrepostas de Camada 2 em cima de redes de camada 3* ... Roda sobre a infraestrutura física existente e prove um meio para “estender” a rede de Camada 2.” [RFC 7348]

Camada de Enlace, LANs: roteiro

- introdução
- detecção, correção de erros
- protocolos de acesso múltiplo
- LANs
 - endereçamento, ARP
 - Ethernet
 - switches
 - VLANs
- virtualização de enlace: MPLS
- redes de datacenter

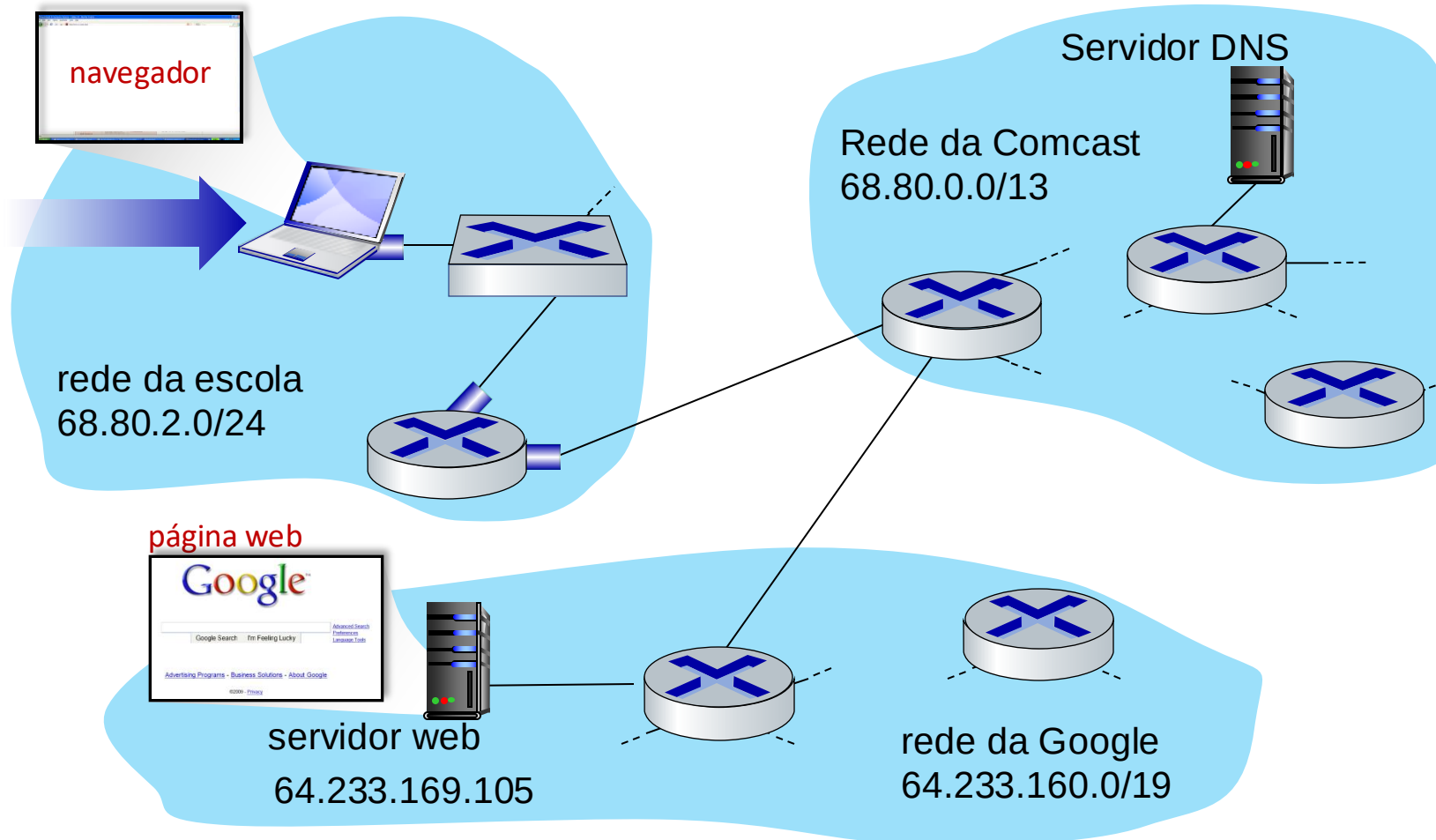


- um dia na vida de uma solicitação web

Síntese: um dia na vida de um pedido web

- completamos agora a nossa jornada descendo na pilha de protocolos!
 - aplicação, transporte, rede, enlace
- colocando tudo junto: síntese!
 - *objetivo*: identificar, revisar, entender os protocolos (em todas as camadas) envolvidos em um cenário aparentemente simples: solicitação de uma página web
 - *cenário*: estudante conecta laptop à rede do campus, solicita/recebe a página www.google.com

Um dia na vida: cenário

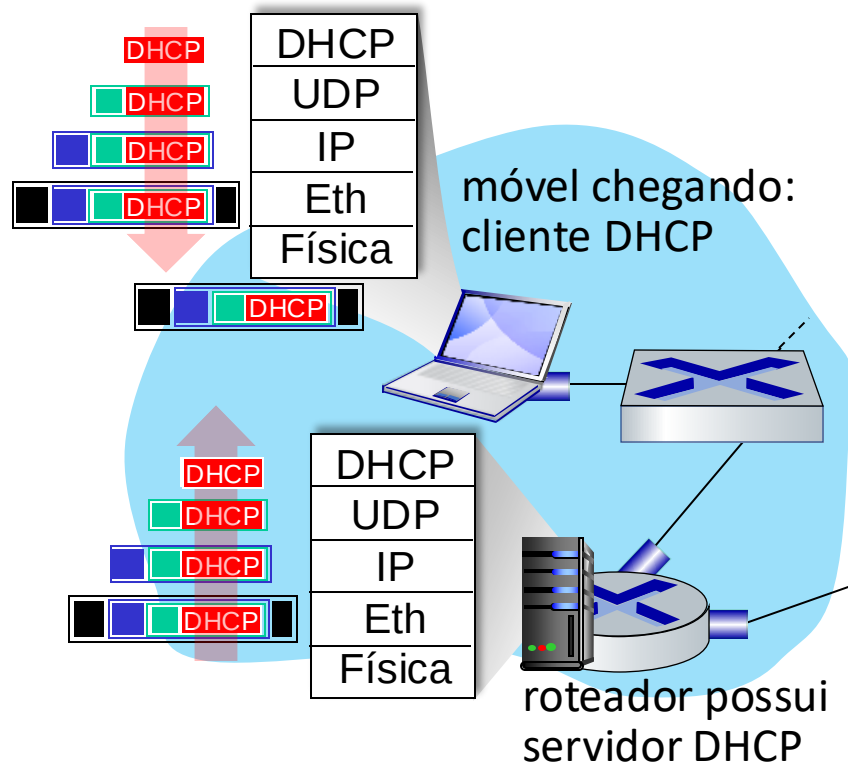


cenário:

- cliente móvel que chega se conecta à rede ...
- solicita página web: www.google.com

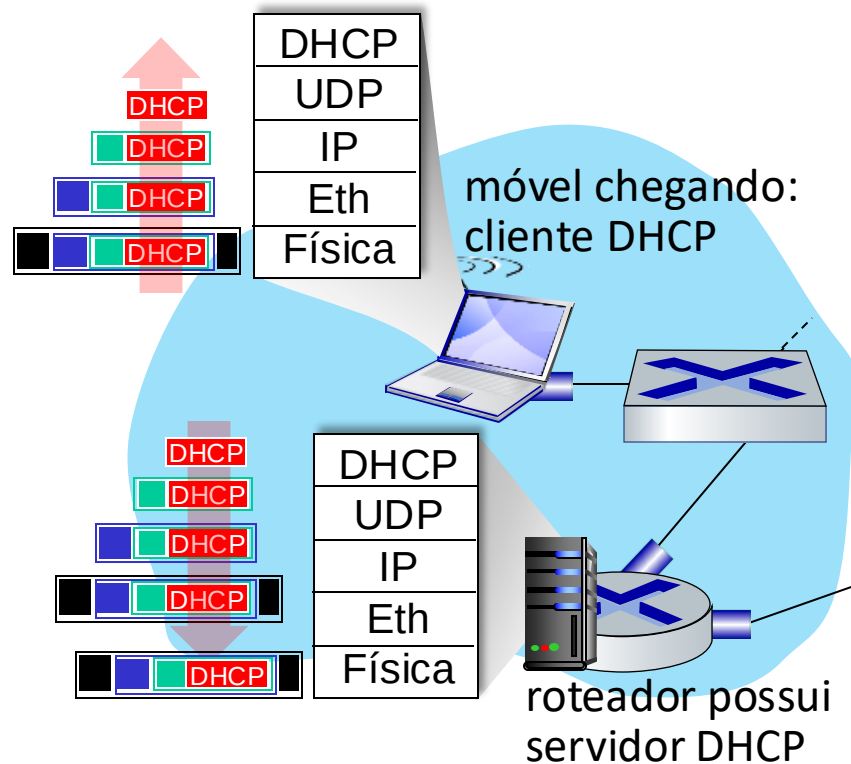
Parece simples! 

Um dia na vida: conectando à Internet



- o laptop que se conecta precisa obter o seu endereço IP, end. do primeiro roteador, end. do servidor DNS: usa **DHCP**
- solicitação DHCP **encapsulada** no **UDP**, encapsulada no **IP**, encapsulada no Ethernet **802.3**
- quadro Ethernet **difundido** (dest: FFFFFFFFFFFFFFFF) na LAN, recebido pelo roteador rodando o servidor **DHCP**
- Ethernet **demultiplexado** para IP, demultiplexado para UDP, demultiplexado para DHCP

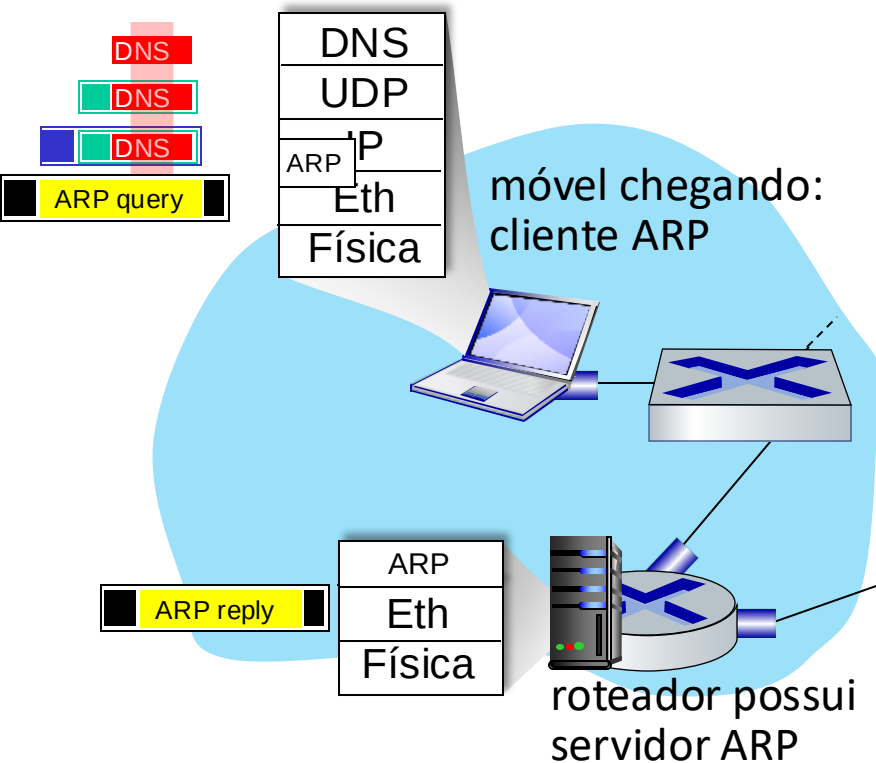
Um dia na vida: conectando à Internet



- servidor DHCP prepara **ACK DHCP** contendo endereço IP do cliente, endereço IP do primeiro roteador, nome e endereço IP do servidor DNS
- encapsulamento no servidor DHCP, quadro repassado (**aprendizado do switch**) através da LAN, demultiplexação no cliente
- cliente DHCP recebe resposta ACK DHCP

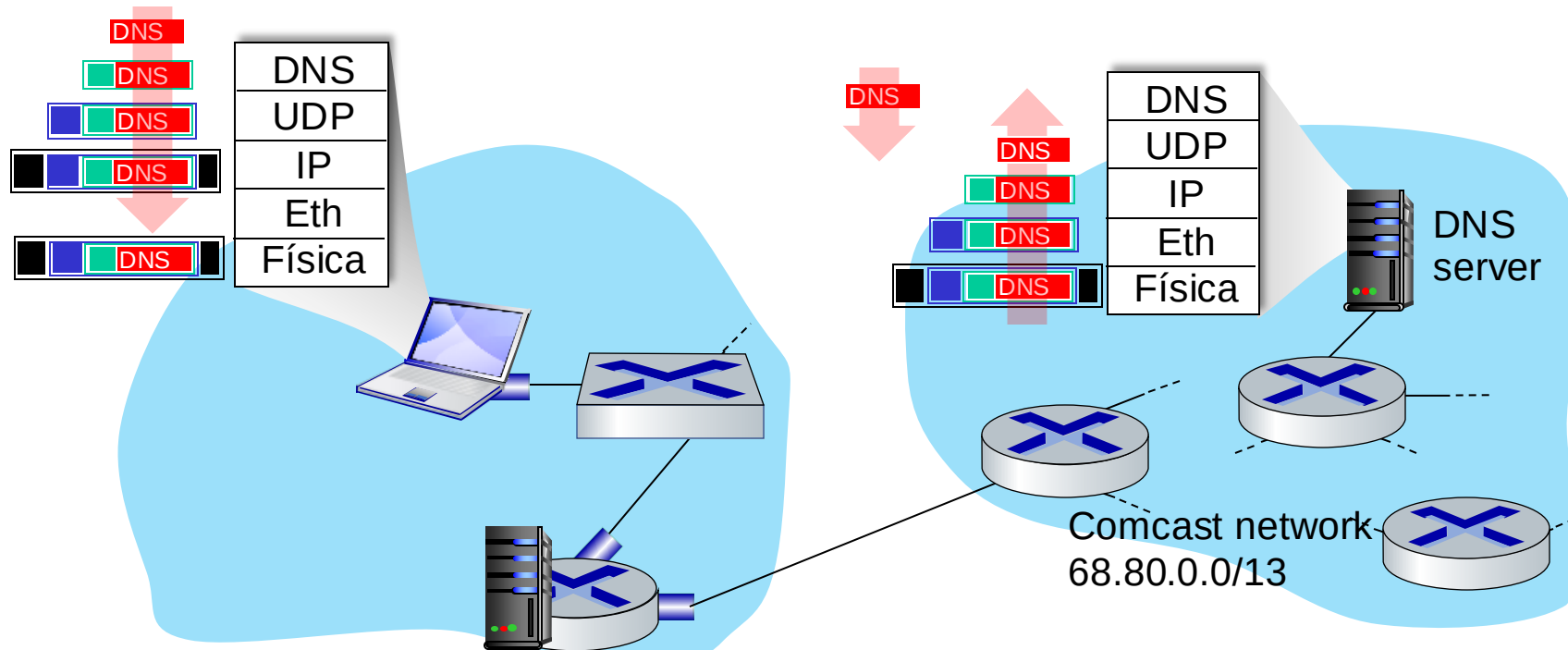
Cliente agora possui um endereço IP, conhece o nome e end. do servidor DNS, e o endereço IP do seu primeiro roteador

Um dia na vida... ARP (antes do DNS, antes do HTTP)



- antes de enviar pedido **HTTP**, necessita o endereço IP de `www.google.com`: **DNS**
- consulta DNS criada, encapsulada no UDP, encapsulada no IP, encapsulada no Eth. Para enviar quadro ao roteador, necessita o endereço MAC da interface do roteador: **ARP**
- **consulta ARP** difundida, recebida pelo roteador, que responde com uma **ARP reply** dando o endereço MAC da interface do roteador
- o cliente agora conhece o endereço MAC do primeiro roteador; podendo agora enviar o quadro contendo a consulta DNS

Um dia na vida... usando DNS

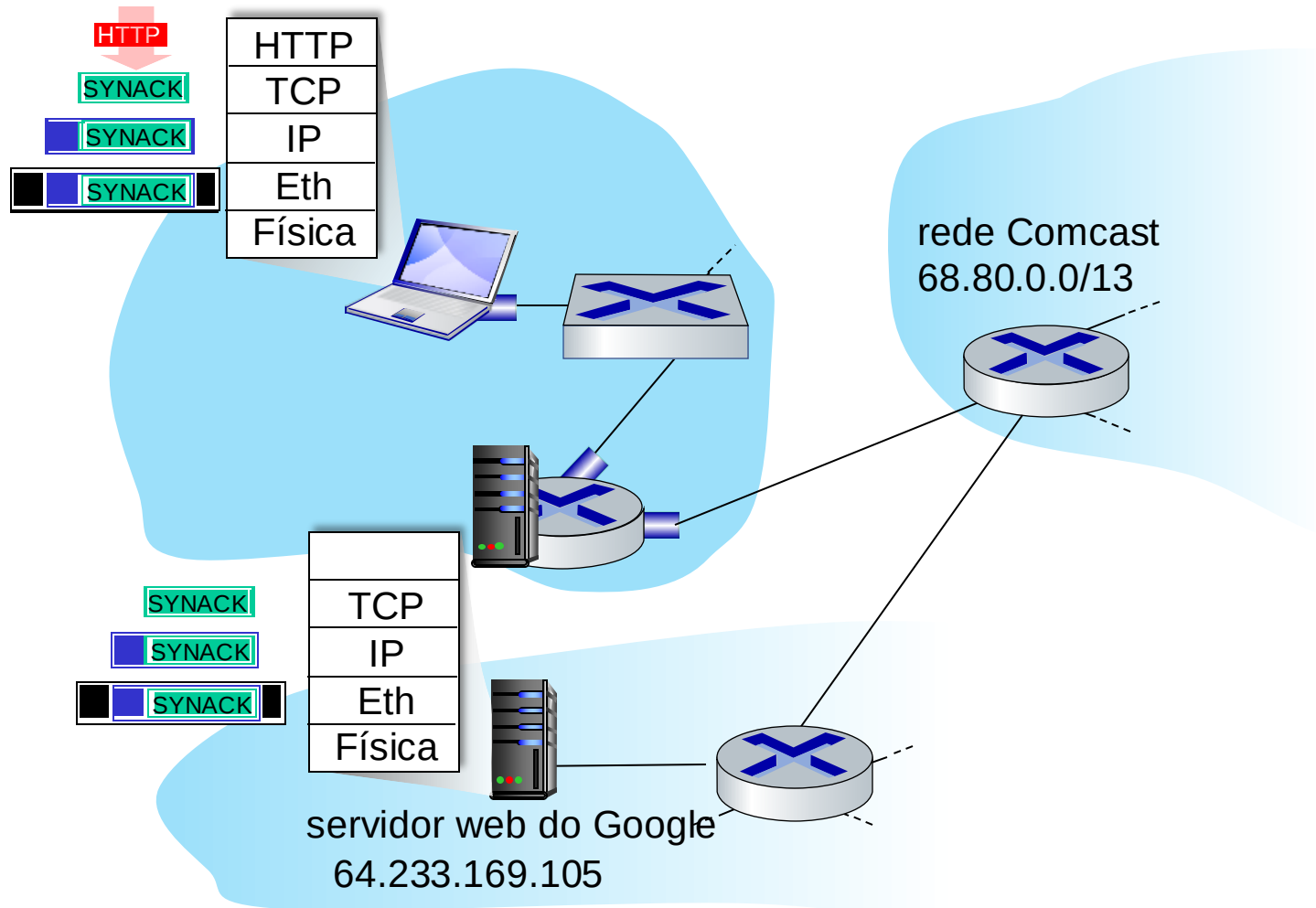


- datagrama IP contém consulta DNS encaminhada através do switch LAN do cliente até o primeiro roteador

- datagrama IP repassado da rede do campus para a rede da Comcast, roteado (tabelas criadas pelos protocolos de roteamento **RIP**, **OSPF**, **IS-IS** e/ou **BGP**) para o servidor DNS

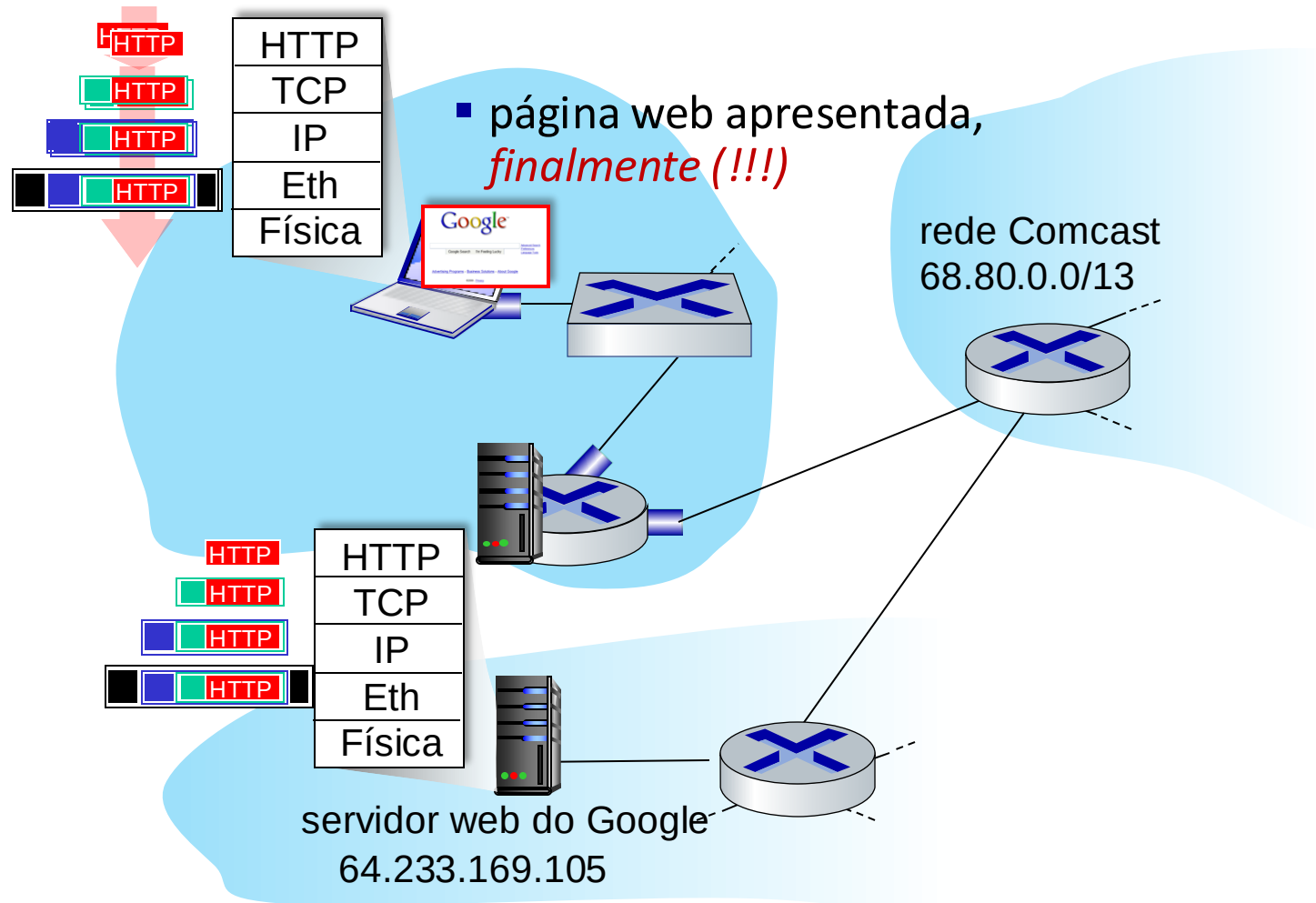
- demultiplexado para o DNS
- servidor DNS responde ao cliente com o endereço IP de www.google.com

Um dia na vida...conexão TCP transportando HTTP



- para enviar pedido HTTP, cliente primeiro abre um **socket TCP** para o servidor web
- **segmento SYN** TCP (passo 1 da saudação em 3 vias) interdomínio roteado para o servidor web
- servidor web responde com **TCP SYNACK** (passo 2 da saudação em 3 vias)
- **conexão TCP estabelecida!**

Um dia na vida... solicitação/resposta HTTP



- **solicitação HTTP** enviada para o socket TCP
- datagrama IP que contém a solicitação HTTP é encaminhado para `www.google.com`
- servidor web responde com **resposta HTTP** (contendo a página web)
- datagrama IP com a resposta HTTP é encaminhado de volta para o cliente

Capítulo 6: Resumo

- princípios por trás dos serviços da camada de enlace de dados:
 - detecção, correção de erros
 - compartilhamento de canal de difusão: acesso múltiplo
 - endereçamento da camada de enlace
- instanciação e implementação de diversas tecnologias de camada de enlace
 - Ethernet
 - LANs comutadas, VLANs
 - redes virtualizadas como camada de enlace: MPLS
- síntese: um dia na vida de uma solicitação web